

57 Secure Digital and MultiMediaCard controller (SDMMC)

57.1 Features

57.1.1 General description

The 3MCR (multi card reader) controller is an advanced host controller compliant with MMC specification version 4.51 and SD Host Controller standard specification version 3.0.

The 3MCR controller handles MMC/SD/SDIO protocols at transmission level, packing data, adding cyclic redundancy check (CRC), start/end bit, and checking for transaction format correctness.

The 3MCR controller provides programmed IO method and DMA data transfer method. In programmed IO method, the host processor transfers data using the Buffer data port register. Host controller supports both SDMA and ADMA2. ADMA2 is enabled by user using programming bit. DMA allows a peripheral to read or write memory without intervention from the CPU. The 3MCR host controller's host controller System address register points to the first data address, and data is then accessed sequentially from that address.

57.1.2 Key features

- Compliance
 - MMC specification version 4.51
 - SD Host Controller standard specification version 3.00
 - SDIO Card specification version 3.0
 - SD Memory Card specification version 3.01
 - SD Memory Card Security specification version 1.01
- System/Host interface
 - Data transfer using PIO mode on the host bus slave interface, using DMA mode on the host bus master interface
 - System/Host interface clock frequency is 100 MHz (max)
- MMC card interface
 - EMMC card clock frequency is 50 MHz
 - Up to 400 Mbits (50 Mbytes) per second data rate using 8 bit parallel data lines (mmc8 bit SDR mode)
 - Up to 800 Mbits (100 Mbytes) per second data rate using 8 bit parallel data lines (mmc8 bit DDR mode)
 - Transfers the data in 1 bit, 4 bit and 8 bit modes and SPI mode
 - Cyclic redundancy check CRC7 for command and CRC16 for data integrity
 - Supports MMC plus and MMC mobile
- SD/ SDIO card interface
 - Host clock rate variable between 0 and 50 MHz
 - Transfers the data in 1 bit and 4 bit SD modes and SPI mode
 - Transfers the data in High and Default speed modes
 - Up to 200 Mbits (25 Mbytes) per second data rate in High speed mode
 - Cyclic Redundancy Check CRC7 for command and CRC16 for data integrity
 - Variable-length data transfers
 - Performs Read Wait control, Suspend/Resume operation SDIO CARD
 - Designed to work with I/O cards, Read Only cards and Read/Write cards
 - Supports Read Wait control, Suspend/Resume operation
- Miscellaneous
 - Handle the FIFO overrun and underrun condition by stopping eMMC card clock

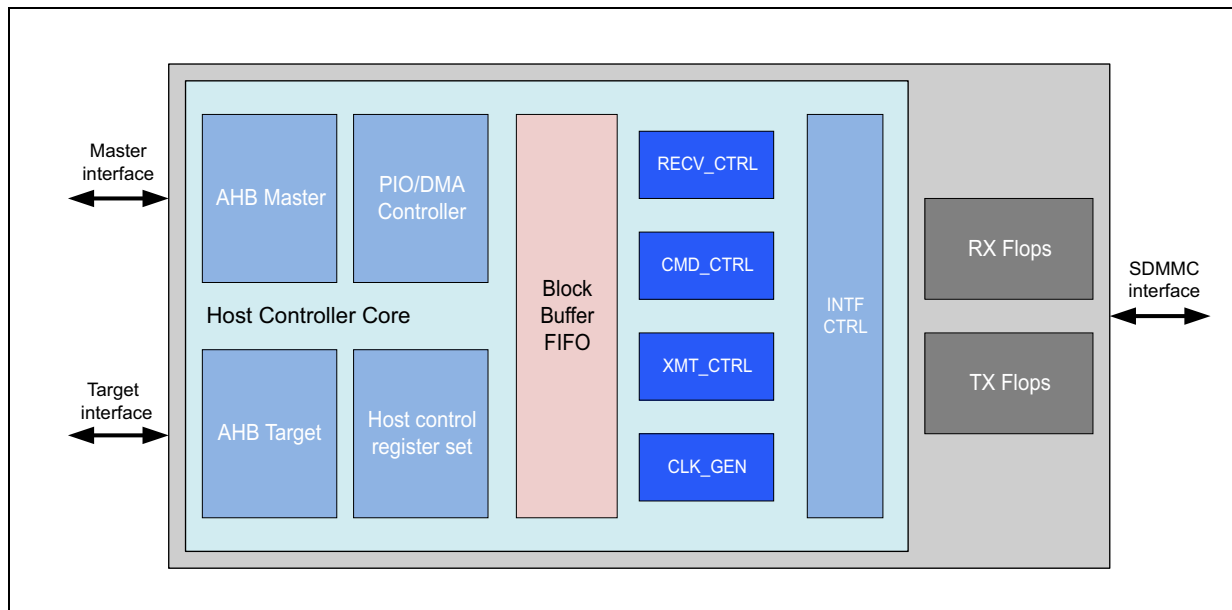
57.2 Architecture

This chapter describes the architecture and block diagram of 3MCR controller.

57.2.1 Block diagram

The block diagram of 3MCR controller is shown in [Figure 1830](#).

Figure 1830. 3MCR controller



57.2.2 Host controller core block description

The 3MCR host controller contains the major part of the design under 3MCR host controller core which contains the following blocks.

57.2.2.1 Host interface (master/target)

The 3MCR host controller interfaces to the system bus using AHB master and AHB target interface.

The target bus is used to access the registers inside the host controller. Also when operating in PIO mode, the driver can access the Data port register through this interface. This is the PIO method in which the host driver transfers data using the Buffer data port register. The target bus supports only single transfer access (no burst support) and only one outstanding read/write transaction.

The master bus is used by the DMA controller (when using SDMA or ADMA2 modes). The DMA controller uses the master DMA interfaces to transfer data between the internal block buffer and the system memory and vice-versa. The DMA controller also uses the master interface to fetch the descriptors while operating in ADMA2 mode.

57.2.2.2 Host controller register set

The host controller registers are dword accessible from the target interface. The host controller register set also implements the Data port register for the PIO mode transfers. The register set provides the control signals to rest of the blocks in design and monitors the status signals from the blocks to set interrupt status bits and eventually generate interrupt signal to the host bus.

The host controller register set acts as the bridge between host CPU and host controller.

The 3MCR controller registers are programmed by the host processor through host target interface. Interrupts are generated to the host processor based on the values set in the Interrupt status and Interrupt enable registers.

SDMMC IP does not provide error response for the accesses to reserved addresses. Also, it does not provide error response for the write accesses to read only registers or vice versa.

57.2.2.3 PIO/DMA controller

The PIO/DMA controller module implements the SDMA and ADMA2 engines as defined in the host controller specification and maintains the block transfer counts for PIO operation. It interacts with the registers set and starts the DMA engine when a command with data transfer is involved.

The DMA controller interfaces to the host master module to generate transfers and on the other side it interfaces with the block buffer to store/fetch block data. The DMA controller implements a separate DMA for SDMA operation and separate DMA for the ADMA2 operation. In addition, it implements host transaction generator that generates controls for the host master interface module.

57.2.2.4 Block buffer

The host controller uses a dual port block buffer (read/write on both ports) that is used to store the block data during data transfers. The size of the block buffer is 1 Kbyte, so that it can accommodate up to two 512B blocks. The block buffer uses circular buffer architecture. One side of the block buffer is interfaced with the DMA controller and operates on the host clock. The other side of the block buffer interfaces with control logic and operates on SDMMC card clock.

During a write transaction (data transferred from system processor to SDMMC card), the data is fetched from the system memory and is stored in the block buffer. When a block of data is available, the control logic will transfer it onto the SDMMC interface. The DMA controller continues to fetch additional block of data when the block buffer has space.

During a read transaction (data transferred from SDMMC card to system processor), the data from SDMMC card will be written in to block buffer and at the end when the CRC of the block is valid, the data is committed. When a block of data is available, then the DMA controller transfers this data to the system memory. The control logic meanwhile receives the next block of data provided there is space in the block buffer. If the host controller cannot accept any data from SDMMC card, then it will stop the data transfer from card by stopping the clock.

Note: As the block buffer size is twice that of the block size, the block buffer behaves like a ping-pong buffer.

57.2.2.5 Clock generator

The clock generator module generates interface clock from the reference clock, based on the controls programmed in the Clock control register. These includes clock divider value and clock enable. The outputs from this module are the SDMMCCTRL_CLK and the SDMMCCARD_CLK. The SDMMCCTRL_CLK is used by the control logic and the SDMMCCARD_CLK connected to the output clock pin of SDMMC interface. This module also generates system resets to various clock domains.

57.2.2.6 Timeout control

The timeout control logic implements the timeout check between block transfers. It uses the contents of the Timeout control register to implement timeout between blocks. This module operates under the control of the transmit control and receive control modules (based on

direction). When the timeout is detected the event is reported to transmit control or receive control module.

57.2.2.7 Command control

The command control module generates the command sequence on the CMD line of the SDMMC interface for every new command programmed by the software.

The command control module also implements the response reception and checking the validity of the response. It uses the response type field to determine the length of the response and the presence of CRC7 field. The response is received on the receive clock (which is the looped back clock). Once the response is received the contents of the response (start bit, command index, CRC7, end bit) are verified and response status is forwarded to the register set module for setting various status bits. It also implements the timeout check on the response reception to make sure that the response is received within the defined time (5 or 64 clocks based on command type). The received response is then stored into appropriate bit position in the Response register.

The command control module generates controls to the transmit control and receive control based on the transfer direction.

The command control module also generates auto command (AutoCMD12 or AutoCMD23) when enabled.

57.2.2.8 Transmit control

The transmit control module is used for write transfers for transferring data to the card. Once the command is issued, this module waits for the block of data to be available in the block buffer and transfers this onto the DAT lines. Based on the configuration of data lines, the (1-bit, 4-bit or 8-bits or SPI), the data from block buffer is appropriately routed. The CRC16 is individually calculated on per lane basis and is attached at the end of block transfer before the end bit. In case of DDR operation, it implements separate CRC16 for each edge of the clock.

At the end of block transfer, it waits for the CRC response on DAT0 line and reports the result of the CRC check to the register set.

Also this module checks for the write busy indication (DAT0 line) before transferring next block of data. The timeout check is implemented to make sure that the write busy is asserted no more than the required limit.

57.2.2.9 Receive control

The receive control module is used for read transfers for receiving data from the card. Once the command is issued, this module waits for the block of data to be received from the card. Based on the configuration of data lines, the (1-bit, 4-bit or 8-bits or SPI), the data from SDMMC interface is assembled into byte and eventually into 32-bit word before it is being written into block buffer. The CRC16 is individually calculated on per lane basis and is checked against the received CRC16 at the end of block transfer before the end bit. In case of DDR operation, it implements separate CRC16 checker for each edge of the clock.

The data is received on the receive clock. This receive clock is the looped back clock (SDMMC_CARD_CLK from the IO_BUF).

The timeout check is implemented to make sure that the gap between the block no more than the required limit.

57.2.2.10 Interface control

The interface control block maps the internal signals to the external SDMMC interface and vice versa. Based on the bus width (1/4/8) the internal signals are driven out appropriately.

The inputs from Rx flops module are latched on the rx_clk (looped back clock) and is output to the receive control module for further processing.

57.2.3 Clocks

SDMMC_CLK: This is the main clock provided by CGM. It is used for the SDMMC interface side of the host controller. This clock input is used to generate sdmmcctrl_clk and sdmmccard_clk clocks based on the divisor value programmed by the host driver.

ahb_clk (commonly called as host_clk): host bus clock. The host bus interface modules, the register set and the DMA controller operate on this clock.

sdmmcctrl_clk: This clock is derived from SDMMC_CLK based on the Clock control register value programmed by the driver. This clock is used by the most of the control logic (command control, transmit control and sdmmc interface side of the block buffer).

sdmmccard_clk: same as sdmmcctrl_clk, except that this is available only when clock enable is set by the driver. This is the clock supplied to the card.

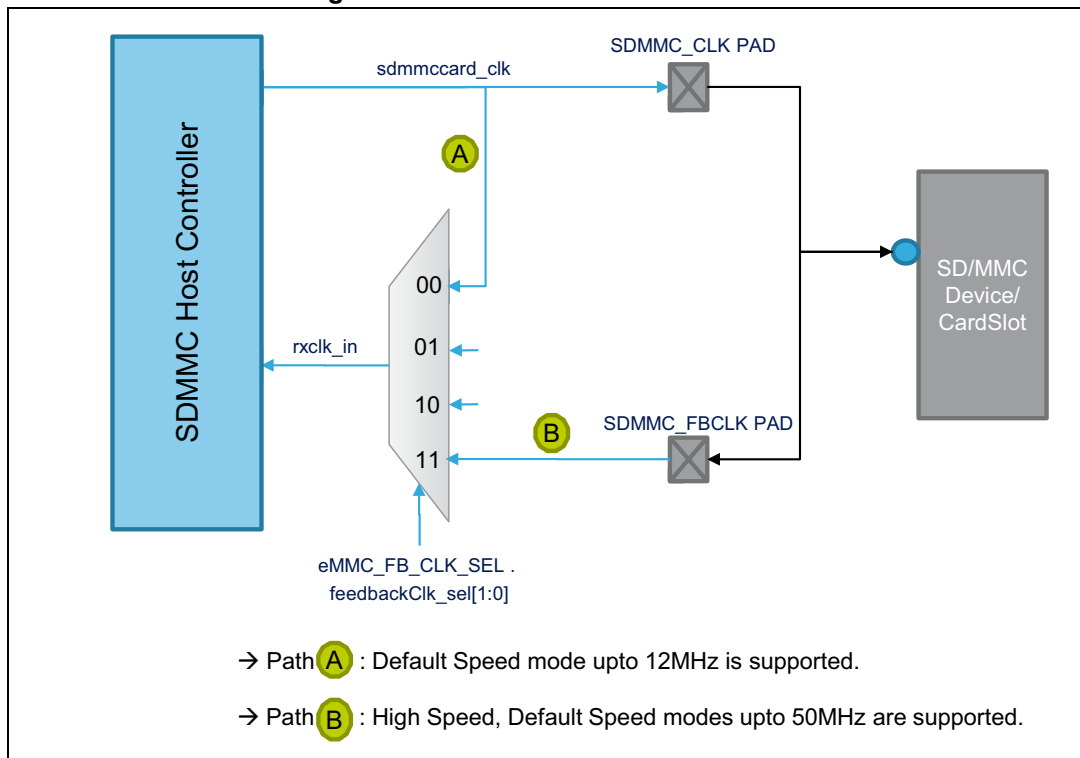
The host controller supports both Full speed and High speed card. For High speed card, the host controller should clock out the data at rising edge of sdmmcctrl_clk and for Full speed card, the host controller should clock out the data at falling edge of sdmmcctrl_clk.

During read transaction when the read FIFOs are full, there is no space to accept one block of data from card, host controller will stop the clock to card so overrun condition will not occur at host controller side.

rxclk_in: This is the looped back clock either from the sdmmccard_clk or from the clk PAD of external memory device mounted on application board. This will account for the delay on the CLK being driven onto the SDMMC interface through the chip and the IO PAD. Refer to figure below for feedback clock selection.

Note: To select the looped back clock software driver should program **FEEDBACKCLK_SEL** field of Feedback clock selection register (**FB_CLK_SEL**).

Figure 1831. Feedback clock selection



57.2.4 Resets

The host controller uses asynchronous reset to reset the FF's, even though the reset itself is synchronous to the corresponding clock domain. `ahb_reset_n` is the primary reset input and is synchronized with corresponding clock domain. Internally an independent reset signal is generated for each of the clock domain from `ahb_reset_n`.

57.2.5 Frequency range

The `host_clk` frequency is 100 MHz. For selection of clock frequency refer to Clocking chapter.

The `sdmmcctrl_clk` frequency range is 400 KHz to 50 MHz.

57.2.6 FIFO overrun and underrun conditions in DMA and non-DMA transfers

57.2.6.1 Write

During write transaction the host controller will transmit data to card only when a block of data is ready to transmit and also the card is not driving busy. So underrun condition will not occur on SDMMC host controller side.

In DMA mode host controller initiates DMA READ from host memory only if space is available to accept a block of data.

In non-DMA mode host controller will assert buffer write ready interrupt only if space is available to accept a block of data.

57.2.6.2 Read

During read transaction, when the internal block buffer is full it can not accept any more data from the card. In these circumstances, host controller will stop the clock to card in order to avoid buffer overrun condition.

In DMA mode host controller initiates DMA WRITE to host memory only on reception of block of data from card.

In non-DMA mode host controller will assert buffer read ready interrupt only on reception of block of data from card.

57.3 Registers

57.3.1 Registers memory map

Base addresses are given in the Memory map chapter.

Note: Reserved fields are read only.

Table 1808. SDMMC memory map

Offset address	Register	Location
IP registers		
0x0000	SDMA System address / Argument 2 register (SDMASYSADDR)	Section 57.3.2.2
0x0004	Block size register (BLOCKSIZE)	Section 57.3.2.3
0x0006	Block count register (BLOCKCOUNT)	Section 57.3.2.4
0x0008	Argument 1 register (ARGUMENT1)	Section 57.3.2.5
0x000C	Transfer mode register (TRANSFERMODE)	Section 57.3.2.6
0x000E	Command register (COMMAND)	Section 57.3.2.7
0x0010 + n*0x2	Response n register (RESPONSEn) for n=0 to 7	Section 57.3.2.8
0x0020	Data port register (DATAPORT)	Section 57.3.2.9
0x0024	Present state register (PRESENTSTATE)	Section 57.3.2.10
0x0028	Host control 1 register (HOSTCONTROL1)	Section 57.3.2.11
0x0029	Power control register (POWERCONTROL)	Section 57.3.2.12
0x002A	Block gap control register (BLOCKGAPCONTROL)	Section 57.3.2.13
0x002B	Wake up control register (WAKEUPCONTROL)	Section 57.3.2.14
0x002C	Clock control register (CLOCKCONTROL)	Section 57.3.2.15
0x002E	Timeout control register (TIMEOUTCONTROL)	Section 57.3.2.16
0x002F	Software reset register (SOFTWARERESET)	Section 57.3.2.17
0x0030	Normal interrupt status register (NORMALINTRSTS)	Section 57.3.2.18
0x0032	Error interrupt status register (ERRORINTRSTS)	Section 57.3.2.19
0x0034	Normal interrupt status enable register (NORMALINTRSTSENA)	Section 57.3.2.20

Table 1808. SDMMC memory map (continued)

Offset address	Register	Location
0x0036	Error interrupt status enable register (ERRORINTRSTSENA)	Section 57.3.2.21
0x0038	Normal interrupt signal enable register (NORMALINTRSIGENA)	Section 57.3.2.22
0x003A	Error interrupt signal enable register (ERRORINTRSIGENA)	Section 57.3.2.23
0x003C	Auto CMD Error status enable register (AUTOCMDERRSTS)	Section 57.3.2.24
0x003E	Host control 2 register (HOSTCONTROL2)	Section 57.3.2.25
0x0040	Capabilities[31:0] register (CAPABILITIES31_0)	Section 57.3.2.26
0x0044	Capabilities[63:32] register (CAPABILITIES63_32)	Section 57.3.2.27
0x0048	Maximum current capabilities[31:0] register (MAXCURRENTCAP31_0)	Section 57.3.2.28
0x004C	Maximum current capabilities[63:32] register (MAXCURRENTCAP63_32)	Section 57.3.2.29
0x0050	Force event for auto CMD error status register (FORCEEVENTFORAUTOCMDERRSTS)	Section 57.3.2.33
0x0052	Force event for error interrupt status register (FORCEEVENTFORERRINTSTS)	Section 57.3.2.34
0x0054	ADMA Error status register (ADMAERRSTS)	Section 57.3.2.35
0x0055–0x0057	Reserved	
0x0058 + n*0x2	ADMA System address n register (ADMASYSADDRn) for n = 0 to 3	Section 57.3.2.36
0x0060 + n*0x2	Preset value n register (PRESETVALUEn) for n = 0 to 7	Section 57.3.2.30
0x0070	Boot timeout control register (BOOTTIMEOUTCNT)	Section 57.3.2.37
0x0074–0x00FB	Reserved	
0x00FC	Slot interrupt status register (SLOTINTRSTS)	Section 57.3.2.31
0x00FE	Host controller version register (HOSTCONTROLLERVER)	Section 57.3.2.32
0x0100–0x01FF	Reserved Note: accesses to this memory range can cause system hang.	
Wrapper registers		
0x0200–0x0207	Reserved	
0x0208	Core configuration register (CORE_CONFIG)	Section 57.3.2.38
0x0210	Feedback clock selection register (FB_CLK_SEL)	Section 57.3.2.39
0x0220	Debug status 1 register (DBG_STA1)	Section 57.3.2.40
0x0224	Debug status 2 register (DBG_STA2)	Section 57.3.2.41
0x0228	Debug status 3 register (DBG_STA3)	Section 57.3.2.42

Table 1808. SDMMC memory map (continued)

Offset address	Register	Location
0x022C	Debug status 4 register (DBG_STA4)	Section 57.3.2.43
0x0230	Debug status 5 register (DBG_STA5)	Section 57.3.2.44

57.3.2 Registers description

The SDMMC host controller registers are described in this section.

The registers to generate SDMMC command are listed in the following table.

Table 1809. Registers to generate SDMMC command

	SDMA command	ADMA command	CPU data transfer	Non-DAT transfer
SDMA system No/AutoCMD23 Address/Argument 2	Yes/No	No/Auto CMD23	No/Auto CMD23	No/No
Block size	Yes	Yes	Yes	No (Protected)
Block count	Yes	Yes	Yes	No (Protected)
Argument 2	Yes	Yes	Yes	No (Protected)
Transfer mode	Yes	Yes	Yes	No (Protected)
Command	Yes	Yes	Yes	Yes

The table above shows register settings for three transactions: SDMA generated transactions, ADMA generated transactions and CPU data transfers and non-DAT transfers. When initiating a transaction, the host driver should program these registers sequentially from 0x000 to 0x00F. The beginning register offset may be calculated based on the type of transaction. The last written offset is always 0x00F because writing to the upper byte of the Command register triggers issuance of the command.

57.3.2.1 Register access method

SDMMC registers are defined in little endian address space.

To access these registers, using a big endian processor, byte locations in word and halfword addressed registers should be swapped.

Dedicated hardware is provided for this. The hardware will be enabled using EMMC_SLV_EN_BYTE_SWAP (bit 26) of PCM REG3 SoC Configuration register.

Reset value of EMMC_SLV_EN_BYTE_SWAP (bit 26) is set that is by default swapping is enabled.

57.3.2.2 SDMA System address / Argument 2 register (SDMASYSADDR)

This register contains the physical system memory address used for DMA transfers or the second argument for the Auto CMD23.

Offset: 0x0000

Access: Read/Write

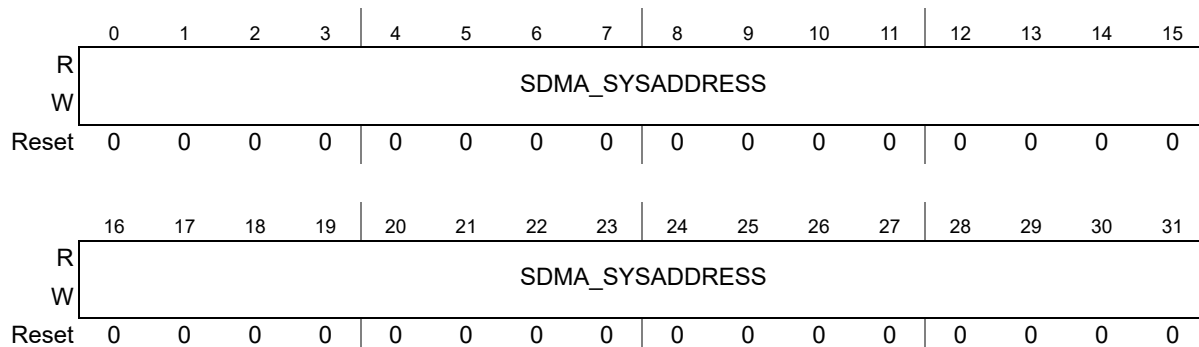


Figure 1832. SDMA System address / Argument 2 register (SDMASYSADDR)

Table 1810. SDMASYSADDR field descriptions

Field	Description
0:31 SDMA_SYSADDRESS	SDMA system address (Low) / Argument 2 (Low) (1) SDMA system address This register contains the system memory address for a SDMA transfer. When the host controller stops a SDMA transfer, this register points to the system address of the next contiguous data position. It can be accessed only if no transaction is executing (after a transaction has stopped). Read operations during transfers may return an invalid value. The host driver initializes this register before starting a SDMA transaction. After SDMA has stopped, the next system address of the next contiguous data position can be read from this register. The SDMA transfer waits at the every boundary specified by the host SDMA buffer boundary in the Block size register. The host controller generates DMA Interrupt to request the host driver to update this register. The host driver sets the next system address of the next data position to this register. When the most upper byte of this register (0x003) is written, the host controller restarts the SDMA transfer. When restarting SDMA by the Resume command or by setting Continue request in the Block gap control register, the host controller starts at the next contiguous address stored here in the SDMA System address register. ADMA does not use this register. (2) Argument 2 This register is used with the Auto CMD23 to set a 32-bit block count value to the argument of the CMD23 while executing Auto CMD23. If Auto CMD23 is used with ADMA, the full 32-bit block count value can be used. If Auto CMD23 is used without AMDA, the available block count value is limited by the Block count register. 65535 blocks is the maximum value in this case.

57.3.2.3 Block size register (BLOCKSIZE)

The block size register is described below.

Offset: 0x0004

Access: Read/Write

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	SDMA_			XFER_BLOCKSIZE											
W		BUFBOUNDARY														
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1833. Block size register (BLOCKSIZE)

Table 1811. BLOCKSIZE field descriptions

Field	Description
1:3 SDMA_ BUFBOUNDARY	Host SDMA buffer size To perform long DMA transfer, System address register is updated at every system boundary during DMA transfer. These bits specify the size of contiguous buffer in the system memory. The DMA transfer waits at the every boundary specified by these fields and the HC generates the DMA Interrupt to request the HD to update the System address register. These bits support when the DMA support in the Capabilities register is set to 1 and this function is active when the DMA enable in the Transfer mode register is set to 1. 000 4 Kbytes (Detects A11 Carry out) 001 8 Kbytes (Detects A12 Carry out) 010 16 Kbytes (Detects A13 Carry out) 011 32 Kbytes (Detects A14 Carry out) 100 64 Kbytes (Detects A15 Carry out) 101 128 Kbytes (Detects A16 Carry out) 110 256 Kbytes (Detects A17 Carry out) 111 512 Kbytes (Detects A18 Carry out)
4:15 XFER_BLOCKSIZE	Transfer block size This register field specifies the block size for block data transfers for CMD17, CMD18, CMD24, CMD25, and CMD53. It can be accessed only if no transaction is executing (that is after a transaction has stopped). Read operations during transfer return an invalid value and Write operations are ignored. 0x0000 No data transfer 0x0001 1 Byte 0x0002 2 Bytes 0x0003 3 Bytes 0x0004 4 Bytes ... 0x01FF 511 Bytes 0x0200 512 Bytes

57.3.2.4 Block count register (BLOCKCOUNT)

The block count register is described below.

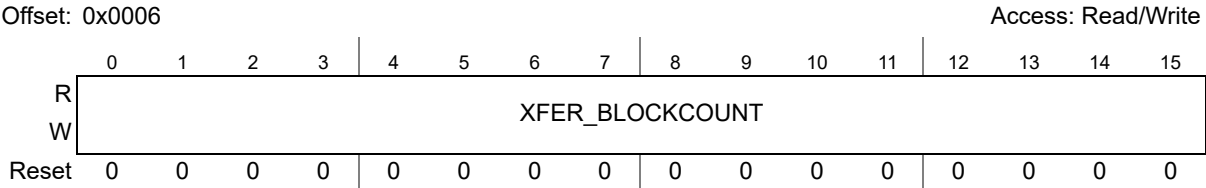


Figure 1834. Block count register (BLOCKCOUNT)

Table 1812. BLOCKCOUNT field descriptions

Field	Description
0:15 XFER_BLOCKCOUNT	Block count for current transfer This register is enabled when Block count enable in the Transfer mode register is set to 1 and is valid only for multiple block transfers. The HC decrements the block count after each block transfer and stops when the count reaches zero. It can be accessed only if no transaction is executing (that is after a transaction has stopped). Read operations during transfer return an invalid value and Write operations are ignored. When saving transfer context as a result of Suspend command, the number of blocks yet to be transferred can be determined by reading this register. When restoring transfer context prior to issuing a Resume command, the HD restores the previously save block count. 0x0000 Stop count 0x0001 1 block 0x0002 2 blocks ... 0xFFFF 65535 blocks

57.3.2.5 Argument 1 register (ARGUMENT1)

The argument 1 register is described below.

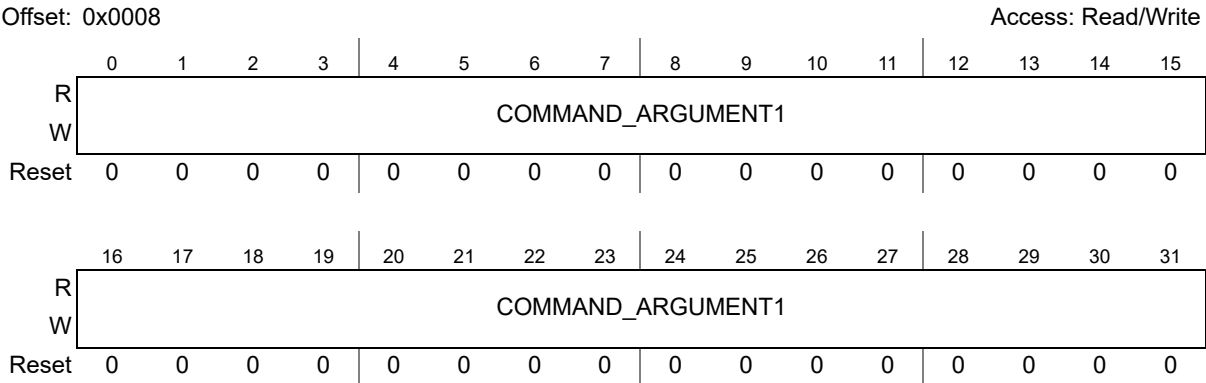


Figure 1835. Argument 1 register (ARGUMENT1)

Table 1813. ARGUMENT1 field descriptions

Field	Description
0:31 COMMAND_ARGUMENT1	Command argument 1 The MMC command argument is specified as bit39-8 of Command-Format.

57.3.2.6 Transfer mode register (TRANSFERMODE)

The transfer mode register is described below.

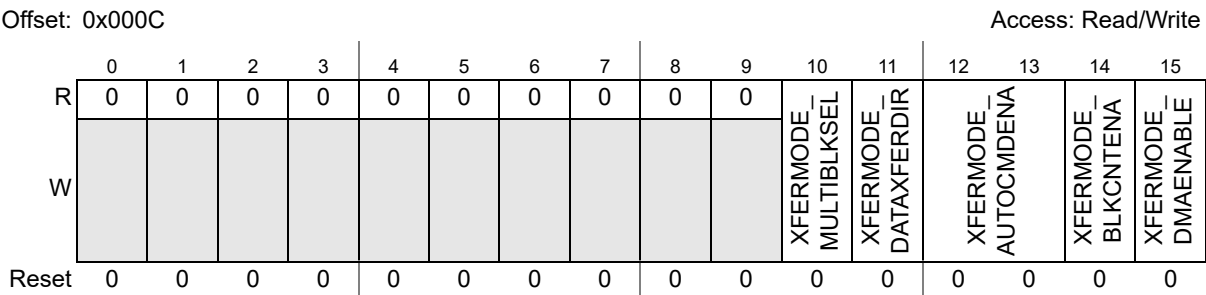


Figure 1836. Transfer mode register (TRANSFERMODE)

Table 1814. TRANSFERMODE field descriptions

Field	Description
10 XFERMODE_	Multi/Single block select This bit enables multiple block data transfers. 0 Single block 1 Multiple block
11 XFERMODE_	Data transfer direction select This bit defines the direction of data transfers. 0 Write (host to card) 1 Read (card to host)

Table 1814. TRANSFERMODE field descriptions (continued)

Field	Description
12:13 XFERMODE_ AUTOCMDENA	Auto CMD enable This field determines use of auto command functions. 00 Auto command disabled 01 Auto CMD12 enable 10 Auto CMD23 enable 11 Reserved There are two methods to stop multiple-block read and write operation. (1) Auto CMD12 enable Multiple-block read and write commands for memory require CMD12 to stop the operation. When this field is set to 01b, the host controller issues CMD12 automatically when last block transfer is completed. Auto CMD12 error is indicated to the Auto CMD Error status register. The host driver does not set this bit if the command does not require CMD12. (2) Auto CMD23 enable When this bit field is set to 10b, the host controller issues a CMD23 automatically before issuing a command specified in the Command register. The following conditions are required to use the Auto CMD23: A memory card that supports CMD23 (SCR[33]=1). If DMA is used, it is ADMA. Only when CMD18 or CMD25 is issued. By writing the Command register, the host controller issues a CMD23 first and then issues a command specified by the command index in Command register 32-bit block count value for CMD23 is set to SDMA System address / Argument 2 register.
14 XFERMODE_ BLKCNTENA	Block count enable This bit is used to enable the Block count register, which is only relevant for multiple block transfers. When this bit is 0, the Block count register is disabled, which is useful in executing an infinite transfer. 0 Disable 1 Enable
15 XFERMODE_ DMAENABLE	DMA enable DMA can be enabled only if DMA support bit in the Capabilities register is set. If this bit is set to 1, a DMA operation begins when the HD writes to the upper byte of Command register (0x00F). 0 Disable 1 Enable

Table 1815. Determination of transfer type

Multi / Single block	Block count enable	Block count	Function
0	Don't care	Don't care	Single Transfer
1	0	Don't care	Infinite Transfer
1	1	Not Zero	Multiple Transfer
1	1	Zero	Stop Multiple Transfer

57.3.2.7 Command register (COMMAND)

The command register is described below.

Offset: 0x000E

Access: Read/Write

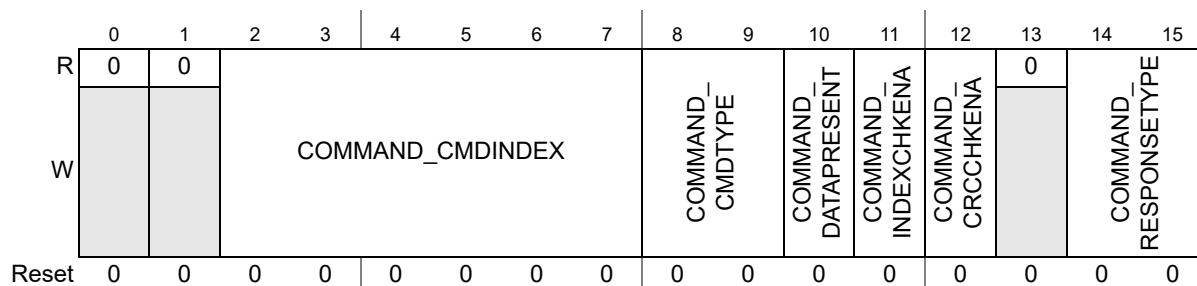


Figure 1837. Command register (COMMAND)

Table 1816. COMMAND field descriptions

Field	Description
2:7 COMMAND_CMDINDEX	Command index This bit is set to the command number (CMD0–63, ACMD0–63).
8:9 COMMAND_CMDTYPE	Command type There are three types of special commands: suspend, resume and abort. These bits are set to 00b for all other commands. <i>Suspend command</i> If the Suspend command succeeds, the HC assumes the eMMC Bus has been released and that it is possible to issue the next command which uses the DAT line. The HC de-asserts Read Wait for read transactions and stop checking busy for write transactions. The interrupt cycle starts, in 4-bit mode. If the Suspend command fails, the HC maintains its current state and the HD restarts the transfer by setting Continue request in the Block gap control register. <i>Resume command</i> The HD re-starts the data transfer by restoring the registers in the range of 0x000–0x00D. The HC checks for busy before starting write transfers. <i>Abort command</i> If this command is set when executing a read transfer, the HC stops Reads to the buffer. If this command is set when executing a write transfer, the HC stops driving the DAT line. After issuing the Abort command, the HD issues a software reset. 00 Normal 01 Suspend 10 Resume 11 Abort

Table 1816. COMMAND field descriptions (continued)

Field	Description
10 COMMAND_ DATAPRESENT	Data present select This bit is set to 1 to indicate that data is present and is transferred using the DAT line. It is set to 0 for the following: Commands using only CMD line (ex. CMD52) Commands with no data transfer but using busy signal on DAT[0] line (R1b or R5b ex. CMD38) Resume command 0 No data present 1 Data present
11 COMMAND_ INDEXCHKENA	Command index check enable If this bit is set to 1, the HC checks the index field in the response to see if it has the same value as the command index. If it is not, it is reported as a command index error. If this bit is set to 0, the index field is not checked. 0 Disable 1 Enable
12 COMMAND_ CRCCHKENA	Command CRC check enable If this bit is set to 1, the HC checks the CRC field in the response. If an error is detected, it is reported as a command CRC error. If this bit is set to 0, the CRC field is not checked. 0 Disable 1 Enable
14:15 COMMAND_ RESPONSETYPE	Response type select 00 No Response 01 Response length 136 10 Response length 48 11 Response length 48 check. Busy after response.

57.3.2.8 Response n register (RESPONSEn)

The response n register is described below.

Offset: $0x0010 + n \times 0x2$ ($n=0$ to 7)

Access: Read

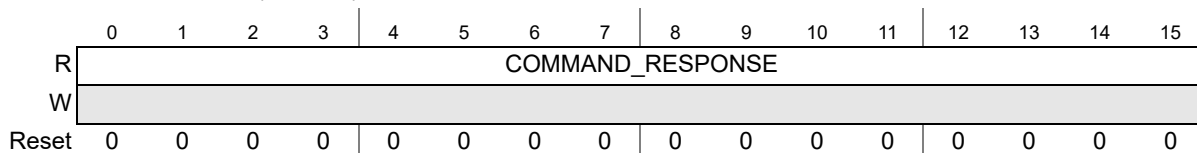


Figure 1838. Response n register (RESPONSEn)

Table 1817. RESPONSEn field descriptions

Field	Description
0:15 COMMAND_RESPONSE	Command response R[] refers to a bit range within the response data as transmitted on the SDMMC bus, REP[] refers to a bit range within the Response register. Note: These bits are read only bits and initialized to zero at reset.

Table 1818 defines the relation between the parameter and name of response type.

Table 1818. Determination of transfer type

Response type	Index check enable	CRC check enable	Name of response type
00	0	0	No response
01	0	1	R2
10	0	0	R3, R4
10	1	1	R1, R6, R5, R7
11	1	1	R1b, R5b

Table 1819 describes the mapping of command responses from the SDMMC bus to this register for each response type. In the table, R[] refers to a bit range within the response data as transmitted on the SDMMC bus, REP[] refers to a bit range within the Response register.

Table 1819. Response bit definition for each response type

Kind of response	Meaning of response	Response field	Response register
R1, R1b (normal response)	Card status	R[39:8]	REP[31:0]
R1b (Auto CMD12 response)	Card status for Auto CMD12	R[39:8]	REP[127:96]
R1 (Auto CMD23 response)	Card status for Auto CMD23	R[39:8]	REP[127:96]
R2 (CID, CSD, register)	CID or CSD reg. incl.	R[127:8]	REP[119:0]
R3 (OCR register)	OCR register for memory	R[39:8]	REP[31:0]

57.3.2.9 Data port register (DATAPORT)

The buffer data port register is described below.

Offset: 0x0020

Access: Read/Write

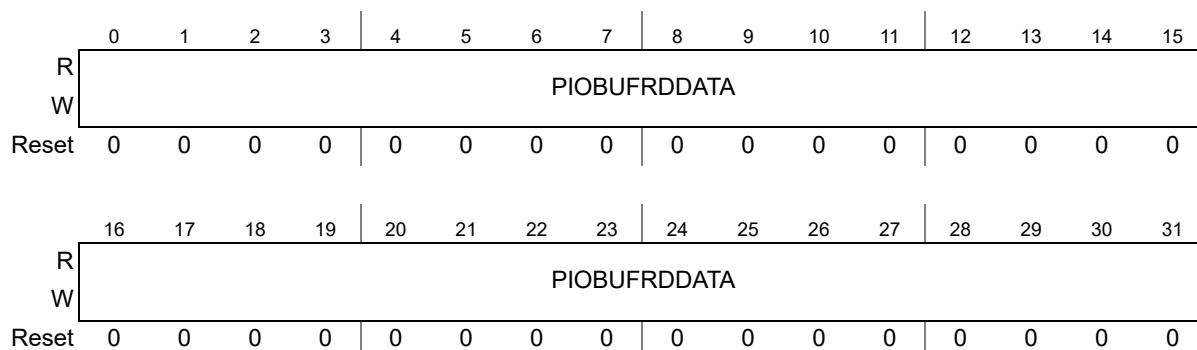


Figure 1839. Data port register (DATAPORT)

Table 1820. DATAPORT field descriptions

Field	Description
0:31 PIOBUFRDDATA	Buffer data The host controller buffer can be accessed through this 32-bit Data port register.

57.3.2.10 Present state register (PRESENTSTATE)

Each bit of this register indicates whether one of the DAT line on SDMMC bus is in use.

Offset: 0x0024

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	SDMMC_DAT7	SDMMC_DAT6	SDMMC_DAT5	SDMMC_DAT4	SDMMC_CMD	SDMMC_DAT3	SDMMC_DAT2	SDMMC_DAT1	SDMMC_DAT0	SDMMC_WP	SDMMC_CD_N	CARDDET_STATESTABLE	CARDDET_INSERTED
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	0	0	0	0	DMACTRL_PIOBUFRDENA	DMACTRL_PIOBUFWRENA	DMACTRL_RDXFERACTIVE	DMACTRL_WRXFERACTIVE	0	0	0	0	0	DMACTRL_DATAINEACTIVE	PRESENTSTATE_INHIBITDAT	PRESENTSTATE_INHIBITCMD
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1840. Present state register (PRESENTSTATE)

Table 1821. PRESENTSTATE field descriptions

Field	Description
3:6 SDMMC_DAT[7:4]	DAT[7:4] line signal level This status is used to check DAT line level to recover from errors, and for debugging.
7 SDMMC_CMD	CMD line signal level This status is used to check CMD line level to recover from errors, and for debugging.
8:11 SDMMC_DAT[3:0]	DAT[3:0] line signal level This status is used to check DAT line level to recover from errors, and for debugging. This is especially useful in detecting the busy signal level from DAT[0].

Table 1821. PRESENTSTATE field descriptions (continued)

Field	Description
12 SDMMC_WP	Write protect switch pin level The write protect switch is supported for memory and combo cards. 0 Write protected 1 Write enabled Note: No write protection is provided. This bit will always read as 1. Ignore this bit because cards are always enabled for writing.
13 SDMMC_CD_N	Card detect pin level 0 No card present 1 Card present
14 CARDDET_STATESTABLE	Card state stable This bit is used for testing. If it is 0, the card detect pin level is not stable. If this bit is set to 1, it means the card detect pin level is stable. The software reset for all in the Software reset register does not affect this bit. 0 Reset of debouncing 1 No card or inserted
15 CARDDET_INSERTED	Card inserted This bit indicates whether a card has been inserted. Changing from 0 to 1 generates a Card Insertion Interrupt in the Normal interrupt status register and changing from 1 to 0 generates a Card Removal Interrupt in the Normal interrupt status register. The software reset for all in the Software reset register does not affect this bit. 0 Reset or debouncing or no card 1 Card inserted
20 DMACTRL_PIOBUFRDENA	Buffer read enable This status is used for non-DMA read transfers. This read only flag indicates that valid data exists in the host side buffer status. If this bit is 1, read-able data exists in the buffer. A change of this bit from 1 to 0 occurs when all the block data is read from the buffer. A change of this bit from 0 to 1 occurs when all the block data is ready in the buffer and generates the Buffer Read Ready Interrupt. 0 Read disable 1 Read enable Note: This bit is read only bit and initialized to zero at reset.
21 DMACTRL_PIOBUFWRENA	Buffer write enable This status is used for non-DMA write transfers. This read only flag indicates if space is available for write data. If this bit is 1, data can be written to the buffer. A change of this bit from 1 to 0 occurs when all the block data is written to the buffer. A change of this bit from 0 to 1 occurs when top of block data can be written to the buffer and generates the buffer write ready interrupt. 0 Write disable 1 Write enable. Note: This bit is read only bit and initialized to zero at reset.

Table 1821. PRESENTSTATE field descriptions (continued)

Field	Description
22 DMACTRL_ RDXFERACTIVE	Read transfer active This status is used for detecting completion of a read transfer. This bit is set to 1 for either of the following conditions: – After the end bit of the read command. – When writing a 1 to Continue request in the Block gap control register to restart a read transfer. This bit is cleared to 0 for either of the following conditions: – When the last data block as specified by block length is transferred to the system. – When all valid data blocks have been transferred to the system and no current block transfers are being sent as a result of the Stop at block gap request set to 1. A Transfer Complete Interrupt is generated when this bit changes to 0. 1 Transferring data 0 No valid data Note: This bit is read only bit and initialized to zero at reset.
23 DMACTRL_ WRXFERACTIVE	Write transfer active This status indicates a write transfer is active. If this bit is 0, it means no valid write data exists in the HC. This bit is set in either of the following cases: – After the end bit of the write command. – When writing a 1 to Continue request in the Block gap control register to restart a write transfer. This bit is cleared in either of the following cases: – After getting the CRC status of the last data block as specified by the transfer count (single or multiple). – After getting a CRC status of any block where data transmission is about to be stopped by a Stop at block gap request. During a write transaction, a block gap event interrupt is generated when this bit is changed to 0, as a result of the Stop at block gap request being set. This status is useful for the HD in determining when to issue commands during write busy. 1 Transferring data 0 No valid data Note: This bit is read only bit and initialized to zero at reset.
29 DMACTRL_ DATA LINEACTIVE	DAT line active This bit indicates whether one of the DAT line on SDMMC bus is in use. 1 DAT line active 0 DAT line inactive Note: This bit is read only bit and initialized to zero at reset.

Table 1821. PRESENTSTATE field descriptions (continued)

Field	Description
30 PRESENTSTATE_ INHIBITDAT	Command inhibit (DAT) This status bit is generated if either the DAT line active or the read transfer active is set to 1. If this bit is 0, it indicates the HC can issue the next command. Commands with busy signal belong to Command inhibit (DAT) (ex. R1b, R5b type). Changing from 1 to 0 generates a Transfer Complete Interrupt in the Normal interrupt status register. Note: The host driver can save registers in the range of 0x000-0x00D for a Suspend transaction after this bit has changed from 1 to 0. 1 Cannot issue command which uses the DAT line 0 Can issue command which uses the DAT line Note: This bit is read only bit and initialized to zero at reset.
31 PRESENTSTATE_ INHIBITCMD	Command inhibit (CMD) If this bit is 0, it indicates the CMD line is not in use and the HC can issue a command using the CMD line. This bit is set immediately after the Command register (0x00F) is written. This bit is cleared when the command response is received. Even if the Command inhibit (DAT) is set to 1, commands using only the CMD line can be issued if this bit is 0. Changing from 1 to 0 generates a Command Complete Interrupt in the Normal interrupt status register. If the HC cannot issue the command because of a command conflict error or because of command not issued by Auto CMD12 error, this bit remains 1 and the Command Complete is not set. Status issuing Auto CMD12 is not read from this bit. Auto CMD12 and Auto CMD23 consist of two responses. In this case, this bit is not cleared by the response of CMD12 or CMD23 but cleared by the response of a read/write command. Status issuing Auto CMD12 is not read from this bit. So if a command is issued during Auto CMD12 operation, host controller manages to issue two commands: CMD12 and a command set by Command register. Note: This bit is read only bit and initialized to zero at reset.

Note: DAT line active indicates whether one of the DAT line is on SDMMC bus is in use.

In the case of Read transactions:

This status indicates whether a read transfer is executing on the SDMMC bus. Changing this value from 1 to 0 generates a Block Gap Event Interrupt in the Normal interrupt status register, as the result of the Stop at block gap request being set.

This bit is set in either of the following cases:

1. After the end bit of the Read command.
2. When writing a 1 to Continue request in the Block gap control register to restart a read transfer.

This bit is cleared in either of the following cases:

1. When the end bit of the last data block is sent from the SDMMC bus to the host controller. In case of ADMA2, the last block is designated by the last transfer of descriptor table.
2. When a read transfer is stopped at the block gap initiated by a Stop at block gap request.

The host controller stops read operation at the start of the interrupt cycle of the next block gap by stopping SDMMC clock.

In the case of Write transactions:

This status indicates that a write transfer is executing on the SDMMC bus. Changing this value from 1 to 0 generate a Transfer Complete Interrupt in the Normal interrupt status register.

This bit is set in either of the following cases:

1. After the end bit of the Write command.
2. When writing to 1 to Continue request in the Block gap control register to continue a write transfer.

This bit is cleared in either of the following cases:

1. When the SDMMC card releases write busy of the last data block. If SDMMC card does not drive busy signal for 8 SDMMC clocks, the host controller considers the card drive “not busy”. In case of ADMA2, the last block is designated by the last transfer of descriptor table.
2. When the SDMMC card releases write busy prior to waiting for write transfer as a result of a Stop at block gap request.

Command with busy:

This status indicates whether a command indicates busy (for example erase command for memory) is executing on the SDMMC bus. This bit is set after the end bit of the command with busy and cleared when busy is de-asserted. Changing this bit from 1 to 0 generate a Transfer Complete Interrupt in the Normal interrupt status register.

Note: The HD can issue cmd0, cmd12, cmd13 (for memory) when the DAT lines are busy during data transfer. These commands can be issued when command inhibit (CMD) is set to zero. Other commands are issued when command inhibit (DAT) is set to zero.

57.3.2.11 Host control 1 register (HOSTCONTROL1)

The following table describes the Host control 1 register.

Offset: 0x0028				Access: Read/Write				
	0	1	2	3	4	5	6	7
R	HOSTCTRL1_CDSIGSELECT		HOSTCTRL1_CDTESTLEVEL		HOSTCTRL1_EXTDATAWIDTH		HOSTCTRL1_DMASELECT	
W	HOSTCTRL1_HIGHSPEEDEN ^A		HOSTCTRL1_DATAWIDTH					
Reset	0	0	0	0	0	0	0	0

Figure 1841. Host control 1 register (HOSTCONTROL1)

Table 1822. HOSTCONTROL1 field descriptions

Field	Description
0 HOSTCTRL1_ CDSIGSELECT	Card detect signal detection This bit selects source for card detection. 1 The card detect test level is selected 0 Card detection pin is selected (for normal use) Note: Card detect signal is tied to always available state
1 HOSTCTRL1_ CDTESTLEVEL	Card detect test level This bit is enabled while the card detect signal selection is set to 1 and it indicates card inserted or not. Generates (card ins or card removal) interrupt when the normal int sts enable bit is set. 1 Card inserted 0 No card
2 HOSTCTRL1_ EXTDATAWIDTH	Extended data transfer width This bit controls 8-bit bus width mode for embedded device. Support of this function is indicated in 8-bit support for embedded device in the Capabilities register. If a device supports 8-bit bus mode, this bit may be set to 1. If this bit is 0, bus width is controlled by data transfer width in the Host control 1 register. 1 8-bit bus width 0 Bus width is selected by data transfer width
3:4 HOSTCTRL1_ DMASELECT	DMA select One of supported DMA modes can be selected. The host driver checks support of DMA modes by referring the Capabilities register. 00 SDMA is selected 01 32-bit address ADMA1 is selected 10 32-bit address ADMA2 is selected 11 Reserved
5 HOSTCTRL1_ HIGHSPEEDENA	High speed enable This bit is optional. Before setting this bit, the HD checks the High speed support in the Capabilities register. If this bit is set to 0 (default), the HC outputs CMD line and DAT lines at the falling edge of the SDMMC clock (up to 25 MHz/ 20 MHz). If this bit is set to 1, the HC outputs CMD line and DAT lines at the rising edge of the SDMMC clock (up to 50 MHz) If preset value enable in the Host control 2 register is set to 1, host driver needs to reset SDMMC clock enable before changing this field to avoid generating clock glitches. After setting this field, the host driver sets SDMMC clock enable again. 1 High speed mode 0 Normal speed mode
6 HOSTCTRL1_ DATAWIDTH	Data transfer width This bit selects the data width of the HC. The HD selects it to match the data width of the SDMMC card. 8 bit mode is selected using Extended data transfer width field of Host control 1 register. 1 4 bit mode 0 1 bit mode

57.3.2.12 Power control register (POWERCONTROL)

The following table describes the power control register.

Offset: 0x0029

Access: Read/Write

	0	1	2	3	4	5	6	7
R	0	0	0	0	PWRCTRL_SDBUSVOLTAGE			SD_BUS_POWER
W								
Reset	0	0	0	0	0	0	0	0

Figure 1842. Power control register (POWERCONTROL)

Table 1823. POWERCONTROL field descriptions

Field	Description
4:6 PWRCTRL_SDBUSVOLTAGE	eMMC bus voltage select By setting these bits, the HD selects the voltage level. Before setting this register, the HD checks the voltage support bits in the Capabilities register. 111 3.3 Flattop 110 Reserved ... 000 Reserved Note: only 3.3V is supported
7 SD_BUS_POWER	Before setting this bit, the SD host driver set SD Bus voltage select. If the HC detects the No card state, this bit is cleared. 1 Power on 0 Power off

57.3.2.13 Block gap control register (BLOCKGAPCONTROL)

The following table describes the block gap control register.

Offset: 0x002A

Access: Read/Write

	0	1	2	3	4	5	6	7
R	BLKGAPCTRL_BOOTACKENA	BLKGAPCTRL_ALTBOOTMODE	BLKGAPCTRL_BOOTENABLE	BLKGAPCTRL_SPIMODE	BLKGAPCTRL_INTERRUPT	BLKGAPCTRL_RDWAITCTRL	0	BLKGAPCTRL_STOPATBLKGAP
W							BLKGAPCTRL_CONTINUE	
Reset	1	0	0	0	0	0	0	0

Figure 1843. Block gap control register (BLOCKGAPCONTROL)

Table 1824. BLOCKGAPCONTROL field descriptions

Field	Description
0 BLKGAPCTRL_BOOTACKENA	To check for the boot acknowledge in boot operation. 1 Wait for boot ack from eMMC card 0 Will not wait for boot ack from eMMC card
1 BLKGAPCTRL_ALTBOOTMODE	To start boot code access in alternative mode. 1 To start alternate boot mode access 0 To stop alternate boot mode access

Table 1824. BLOCKGAPCONTROL field descriptions (continued)

Field	Description
2 BLKGAPCTRL_ BOOTENABLE	To start boot code access. 1 To start boot code access 0 To stop boot code access
3 BLKGAPCTRL_ SPIMODE	SPI mode enable bit. 1 SPI mode 0 SDMMC mode
4 BLKGAPCTRL_ INTERRUPT	This bit is valid only in 4-bit mode of the SDIO card and selects a sample point in the interrupt cycle. Setting to 1 enables interrupt detection at the block gap for a multiple block transfer. If the SD card cannot signal an interrupt during a multiple block transfer, this bit should be set to 0. When the HD detects an SD card insertion, this bit is set according to the CCCR of the SDIO card.
5 BLKGAPCTRL_ RDWAITCTRL	The Read Wait function is optional for SDIO cards. If the card supports Read Wait, set this bit to enable use of the Read Wait protocol to stop read data using DAT[2] line. Otherwise the HC has to stop the SD clock to hold read data, which restricts commands generation. When the HD detects an SD card insertion, this bit is set according to the CCCR of the SDIO card. If the card does not support Read Wait, this bit is never set to 1 otherwise DAT line conflict may occur. If this bit is set to 0, Suspend/Resume cannot be supported 1 Enable Read Wait control 0 Disable Read Wait control
6 BLKGAPCTRL_ CONTINUE	Continue request This bit is used to restart a transaction which was stopped using the Stop at block gap request. To cancel stop at the block gap, set Stop at block gap request to 0 and set this bit to restart the transfer. The HC automatically clears this bit in either of the following cases: 1) In the case of a Read transaction, the DAT Line Active changes from 0 to 1 as a read transaction restarts. 2) In the case of a Write transaction, the Write transfer active changes from 0 to 1 as the write transaction restarts. Therefore it is not necessary for host driver to set this bit to 0. If Stop at block gap request is set to 1, any write to this bit is ignored. 1 Restart 0 Ignored Note: The host driver requests a host controller operation by setting the bit. The host controller clears the bit automatically when the operation is completed. Writing a 0 has no effect.
7 BLKGAPCTRL_ STOPATBLKGAP	Stop at block gap request This bit is used to stop executing a transaction at the next block gap for non-DMA, SDMA and ADMA transfers. Until the transfer complete is set to 1, indicating a transfer completion, the HD leaves this bit set to 1. Clearing both the Stop at block gap request and Continue request do not cause the transaction to restart. In case of write transfers in which the HD writes data to the Buffer data port register, the HD set this bit after all block data is written. If this bit is set to 1, the HD does not write data to Buffer data port register. This bit affects Read transfer active, Write transfer active, DAT line active and Command inhibit (DAT) in the Present state register. 1 Stop 0 Transfer

57.3.2.14 Wake up control register (WAKEUPCONTROL)

This register is mandatory for the HC, but wakeup functionality depends on the HC system hardware and software. The HD maintains voltage on the SD bus, by setting SD bus power to 1 in the Power control register, when wakeup event via card interrupt is desired.

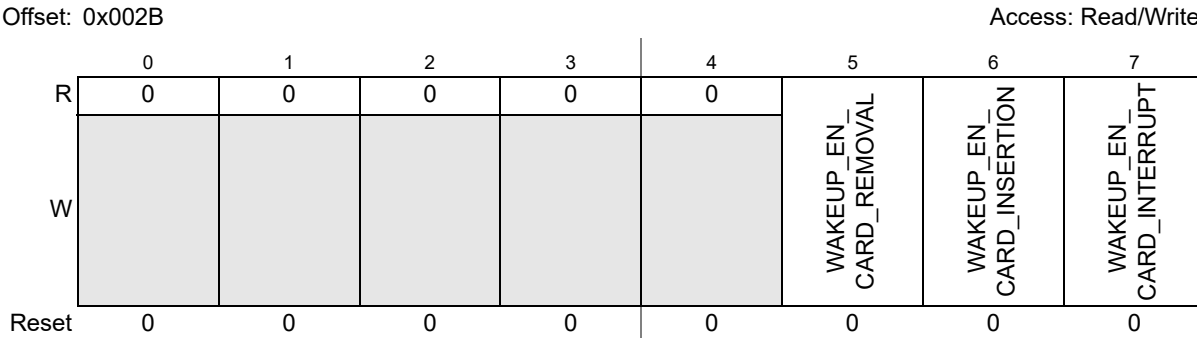


Figure 1844. Wakeup control register (WAKEUPCONTROL)

Table 1825. WAKEUPCONTROL field descriptions

Field	Description
5 WAKEUP_EN_CARD_REMOVAL	Wakeup event enable on SD card removal. This bit enables wakeup event via Card removal assertion in the Normal interrupt status register. 1 Enable 0 Disable
6 WAKEUP_EN_CARD_INSERTION	Wakeup event enable on SD card insertion This bit enables wakeup event via Card insertion assertion in the Normal interrupt status register. 1 Enable 0 Disable
7 WAKEUP_EN_CARD_INTERRUPT	Wakeup event enable on card interrupt This bit enables wakeup event via Card interrupt assertion in the Normal interrupt status register. 1 Enable 0 Disable

57.3.2.15 Clock control register (CLOCKCONTROL)

At the initialization of the HC, the HD set the sdmmcctrl_clk frequency select according to the Capabilities register.

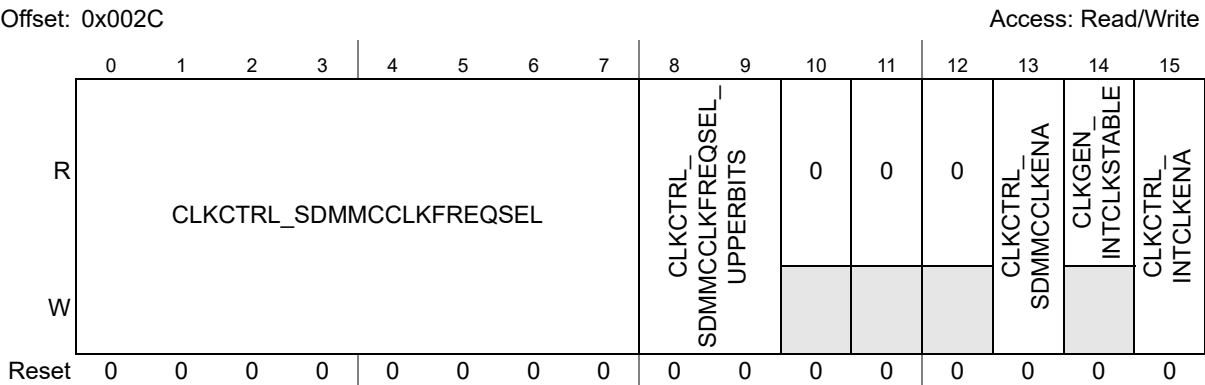


Figure 1845. Clock control register (CLOCKCONTROL)



Table 1826. CLOCKCONTROL field descriptions

Field	Description
0:7 CLKCTRL_ SDMMCCLKFREQSEL	sdmmcctrl_clk frequency select (Lower bits) This register is used to select the frequency of the sdmmcctrl_clk pin. The frequency is not programmed directly; rather this register holds the divisor of the base clock frequency for SDMMC_CLK clock in the Capabilities register. The length of divider is extended to 10 bits and all divider values are supported. 2 upper bits, [9:8], are programmable in clkctrl_sdmmcclkfreqsel_upperbits field. 0x3FF 1/2046 divided clock 0xFFFF 1/2N divided clock (Duty 50%) 0x002 1/4 divided clock 0x001 1/2 divided clock 0x000 Base clock (10 MHz-254 MHz) The three default divider values can be calculated by the frequency that is defined by the Base Clock Frequency for SDMMC clock in the Capabilities register. 400 KHz divider value 25 MHz divider value 50 MHz divider value According to the Physical Layer specification, the maximum SDMMC clock frequency is 25 MHz in Normal speed mode and 50 MHz in High speed mode, and never exceeds this limit. The frequency of SDCLK is set by the following formula: $\text{Clock Frequency} = (\text{Base Clock}) / \text{divisor}$ Thus, choose the smallest possible divisor which results in a clock frequency that is less than or equal to the target frequency. For example, if the base clock frequency for SDMMC clock in the Capabilities register has the value 33 MHz, and the target frequency is 25 MHz, then choosing the divisor value of 01h will yield 16.5 MHz, which is the nearest frequency less than or equal to the target. Similarly, to approach a clock value of 400 KHz, the divisor value of 40h yields the optimal clock value of 258 KHz.
8:9 CLKCTRL_ SDMMCCLKFREQSEL_ UPPERBITS	Upper bits of sdmmcctrl_clk frequency select This bits are assigned to bit [9:8] of clock divider in sdmmcctrl_clk frequency select.
13 CLKCTRL_ SDMMCCLKENA	SDMMC clock enable The HC stops sdmmcctrl_clk when writing this bit to 0. sdmmcctrl_clk frequency select can be changed when this bit is 0. Then, the HC maintains the same clock frequency until sdmmcctrl_clk is stopped. If the HC detects the no card state, this bit is cleared. 1 Enable 0 Disable

Table 1826. CLOCKCONTROL field descriptions (continued)

Field	Description
14 CLKGEN_ INTCLKSTABLE	Internal clock stable This bit is set to 1 when sdmmcctrl_clk clock is stable after writing 1 to Internal clock enable in this register. The SDMMC host driver waits to set sdmmcctrl_clk clock enable until this bit is set to 1. This is useful when using PLL for a clock oscillator that requires setup time. 1 Ready 0 Not ready
15 CLKCTRL_ INTCLKENA	Internal clock enable This bit is set to 0 when the HC is not using the HC or the HC awaits a wakeup event. The HC stops its internal clock to go very low power state. Still, registers may be read and written. Clock starts to oscillate when this bit is set to 1. When clock oscillation is stable, the HC set Internal clock stable in this register to 1. This bit does not affect card detection. 1 Oscillate 0 Stop

57.3.2.16 Timeout control register (TIMEOUTCONTROL)

The following table describes the timeout control register.

At the initialization of the HC, the HD set the data timeout counter value according to the Capabilities register.

Offset: 0x002E Access: Read/Write

	0	1	2	3	4	5	6	7
R	0	0	0	0	TIMEOUT_CTRVALUE			
W								
Reset	0	0	0	0	0	0	0	0

Figure 1846. Timeout control register (TIMEOUTCONTROL)

Table 1827. TIMEOUTCONTROL field descriptions

Field	Description
4:7 TIMEOUT_CTRVALUE	Data timeout counter value This value determines the interval by which DAT line timeouts are detected. Refer to the Data timeout error in the Error interrupt status register for information on factors that dictate timeout generation. Timeout clock frequency will be generated by dividing the TMCLK by this value. When setting this register, prevent inadvertent timeout events by clearing the Data timeout error status enable (in the Error interrupt status enable register) 1111 Reserved 1110 $TMCLK * 2^{27}$... 0001 $TMCLK * 2^{14}$ 0000 $TMCLK * 2^{13}$ Note: TMCLK (Timeout clock) frequency can be either 1 MHz or 1 KHz. Capabilities register, CAPABILITIES31_0, fields CORECFG_TIMEOUTCLKFREQ[5:0] and CORECFG_TIMEOUTCLKUNIT provide the value of TMCLK frequency.

57.3.2.17 Software reset register (SOFTWARERESET)

The following table describes the software reset register.

Note: This register is RW auto clear. The host driver requests a host controller operation by setting the bits. The host controller clears the bits automatically when the operation is completed. Writing a 0 has no effect.

Offset: 0x002F					Access: Read/Write			
	0	1	2	3	4	5	6	7
R	0	0	0	0	0	0	0	0
W						SWRESET_ FOR_ DAT	SWRESET_ FOR_ CMD	SWRESET_ FOR_ ALL
Reset	0	0	0	0	0	0	0	0

Figure 1847. Software reset register (SOFTWARERESET)

Table 1828. SOFTWARERESET field descriptions

Field	Description
5 SWRESET_FOR_DAT	Software reset for DAT line Only part of data circuit is reset. The following registers and bits are cleared by this bit: Buffer data port register: – Buffer is cleared and initialized. Present state register: – Buffer read enable – Buffer write enable – Read transfer active – Write transfer active – DAT line active – Command inhibit (DAT) Block gap control register: – Continue request – Stop at block gap request Normal interrupt status register: – Buffer read ready – Buffer write ready – Block gap event – Transfer complete 1 Reset 0 Work
6 SWRESET_FOR_CMD	Software reset for CMD line Only part of command circuit is reset. The following registers and bits are cleared by this bit: Present state register: – Command inhibit (CMD) Normal interrupt status register: – Command complete 1 Reset 0 Work
7 SWRESET_FOR_ALL	Software reset for all This reset affects the entire HC except for the card detection circuit. Register bits of type R, RW, RW1C, RWAC are cleared to 0. During its initialization, the HD set this bit to 1 to reset the HC. The HC reset this bit to 0 when Capabilities registers are valid and the HD can read them. Additional use of software reset for all may not affect the value of the Capabilities registers. If this bit is set to 1, the SDMMC card resets itself and must be re initialized by the HD. 1 Reset 0 Work

A reset pulse is generated when writing 1 to each bit of this register. After completing the reset, the HC clears each bit. Because it takes some time to complete software reset, the SDMMC host driver confirms that these bits are 0.

57.3.2.18 Normal interrupt status register (NORMALINTRSTS)

The normal interrupt signal enable affects read of this register, but normal interrupt signal does not affect these reads. An interrupt is generated when the normal interrupt signal enable is enabled and at least one of the status bits is set to 1. For all bits except card interrupt and error interrupt, writing 1 to a bit clears it. The card interrupt is cleared when the card stops asserting the interrupt: that is when the card driver services the interrupt condition.

Offset: 0x0030

Access: Read/Write

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	REG_ERRORINTRSTS	NORMALINTRSTS_BOOTCOMPLETE	NORMALINTRSTS_RCVBOOTACK	NORMALINTRSTS_RETUNINGEVENT	NORMALINTRSTS_INTC	NORMALINTRSTS_INTB	NORMALINTRSTS_INTA	NORMALINTRSTS_CARDINTSTS	NORMALINTRSTS_CARDREMSTS	NORMALINTRSTS_CARDINSSTS	NORMALINTRSTS_BUFDRDY	NORMALINTRSTS_BUFWRDY	NORMALINTRSTS_DMAINTERRUPT	NORMALINTRSTS_BLKGAPEVENT	NORMALINTRSTS_XFERCOMPLETE	NORMALINTRSTS_CMDCOMPLETE
W		w1c	w1c						w1c	w1c	w1c	w1c	w1c	w1c	w1c	w1c
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1848. Normal interrupt status register (NORMALINTRSTS)

Table 1829. NORMALINTRSTS field descriptions

Field	Description
0 REG_ERRORINTRSTS	Error interrupt If any of the bits in the Error interrupt status register are set, then this bit is set. Therefore the HD can test for an error by checking this bit first. 0 No error 1 Error
1 NORMALINTRSTS_BOOTCOMPLETE	Boot terminate interrupt This status is set if the boot operation get terminated. 0 Boot operation is not terminated. 1 Boot operation is terminated. Note: SoC does not support Execute In Place from eMMC.
2 NORMALINTRSTS_RCVBOOTACK	Boot ack rcv This status is set if the boot acknowledge is received from device. 0 Boot ack is not received. 1 Boot ack is received. Note: SoC does not support Execute In Place from eMMC.
3 NORMALINTRSTS_RETUNINGEVENT	Re-tuning event This status is set if Re-tuning request in the Present state register changes from 0 to 1. Host controller requests host driver to perform re-tuning for next data transfer. Current data transfer (not large block count) can be completed without re-tuning. 1 Re-tuning is performed. 0 Re-tuning is not required. Note: Re-tuning is not supported.

Table 1829. NORMALINTRSTS field descriptions (continued)

Field	Description
4 NORMALINTRSTS_ INTC	INT_C This status is set if INT_C is enabled and INT_C# pin is in low level. Writing this bit to 1 does not clear this bit. It is cleared by resetting the INT_C interrupt factor. Note: INT_C is not supported.
5 NORMALINTRSTS_ INTB	INT_B This status is set if INT_B is enabled and INT_B# pin is in low level. Writing this bit to 1 does not clear this bit. It is cleared by resetting the INT_B interrupt factor. Note: INT_B is not supported.
6 NORMALINTRSTS_ INTA	INT_A This status is set if INT_A is enabled and INT_A# pin is in low level. Writing this bit to 1 does not clear this bit. It is cleared by resetting the INT_A interrupt factor. Note: INT_A is not supported.
7 NORMALINTRSTS_ CARDINTSTS	Card interrupt Writing this bit to 1 does not clear this bit. 0 No card interrupt 1 Generate card interrupt
8 NORMALINTRSTS_ CARDREMSTS	Card removal This status is set if the card inserted in the Present state register changes from 1 to 0. When the HD writes this bit to 1 to clear this status the status of the card inserted in the Present state register should be confirmed. Because the card detect may possibly be changed when the HD clear this bit an interrupt event may not be generated. 0 Card state stable or debouncing 1 Card removed
9 NORMALINTRSTS_ CARDINSSTS	Card insertion This status is set if the card inserted in the Present state register changes from 0 to 1. When the HD writes this bit to 1 to clear this status, the status of the card inserted in the Present state register should be confirmed. Because the card detect may possibly be changed when the HD clear this bit an interrupt event may not be generated. 0 Card state stable or debouncing 1 Card inserted
10 NORMALINTRSTS_ BUFRDREADY	Buffer read ready This status is set if the Buffer Read Enable changes from 0 to 1. Buffer Read Ready is set to 1 for every CMD19 execution. 0 Not ready to read buffer 1 Ready to read buffer
11 NORMALINTRSTS_ BUFWRREADY	Buffer write ready This status is set if the Buffer Write Enable changes from 0 to 1. 0 Not ready to write buffer 1 Ready to write buffer

Table 1829. NORMALINTRSTS field descriptions (continued)

Field	Description
12 NORMALINTRSTS_ DMAINTERRUPT	DMA interrupt This status is set if the HC detects the host DMA buffer boundary in the Block size register. 0 No DMA interrupt 1 DMA interrupt is generated
13 NORMALINTRSTS_ BLKGAPEVENT	Block gap event If the Stop at block gap request in the Block gap control register is set, this bit is set. Read transaction: This bit is set at the falling edge of the DAT line active status (when the transaction is stopped at SDMMC bus timing. The Read Wait must be supported in order to use this function). Write transaction: This bit is set at the falling edge of write transfer active status (after getting CRC status at SDMMC bus timing). 0 No block gap event 1 Transaction stopped at block gap
14 NORMALINTRSTS_ XFERCOMPLETE	Transfer complete This bit is set when a Read/Write transaction is completed. Read transaction: This bit is set at the falling edge of read transfer active status. There are two cases in which the interrupt is generated. The first is when a data transfer is completed as specified by data length (after the last data has been read to the host system). The second is when data has stopped at the block gap and completed the data transfer by setting the Stop at block gap request in the Block gap control register (after valid data has been read to the host system). Write transaction: This bit is set at the falling edge of the DAT line active status. There are two cases in which the interrupt is generated. The first is when the last data is written to the card as specified by data length and busy signal is released. The second is when data transfers are stopped at the block gap by setting Stop at block gap request in the Block gap control register and data transfers completed. (after valid data is written to the card and the busy signal is released). Note: Transfer complete has higher priority than data timeout error. If both bits are set to 1, the data transfer can be considered complete. 0 No data transfer complete 1 Data transfer complete
15 NORMALINTRSTS_ CMDCOMPLETE	Command complete This bit is set when we get the end bit of the command response (except Auto CMD12 and Auto CMD23). Note: Command timeout error has higher priority than command complete. If both are set to 1, it can be considered that the response was not received correctly. 0 No command complete 1 Command complete

Table 1830. Relation between transfer complete and data timeout error

Transfer complete	Data timeout error	Meaning of the status
0	0	Interrupted by another factor
0	1	Timeout occur during transfer
1	Don't Care	Data transfer complete

Table 1831. Relation between command complete and command timeout error

Command complete	Command timeout error	Meaning of the status
0	0	Interrupted by another factor
Don't care	1	Response not received within 64 sdmmcctrl_clk cycles
1	0	Response received

57.3.2.19 Error interrupt status register (ERRORINTRSTS)

Status defined in this register can be enabled by the Error interrupt status enable register, but not by the Error interrupt signal enable register. The interrupt is generated when the error interrupt signal enable is enabled and at least one of the statuses is set to 1. Writing to 1 clears the bit and writing to 0 keeps the bit unchanged. More than one status can be cleared at the one register write.

Offset: 0x0032

Access: Read/Write

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	ERRORINTRSTS_HO STERROR	0	0	ERRORINTRSTS_A DMAERROR	ERRORINTRSTS_A UTOCMDERROR	0	ERRORINTRSTS_D ATAENDBITERROR	ERRORINTRSTS_D ATACRCERROR	ERRORINTRSTS_D ATATIMEOUTERROR	ERRORINTRSTS_C MDINDEXERROR	ERRORINTRSTS_C MDENDBITERROR	ERRORINTRSTS_C MDCRCERROR	ERRORINTRSTS_C MDTIMEOUTERROR
W				w1c			w1c	w1c		w1c	w1c	w1c	w1c	w1c	w1c	w1c
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1849. Error interrupt status register (ERRORINTRSTS)

Table 1832. ERRORINTRSTS field descriptions

Field	Description
3 ERRORINTRSTS_ HOSTERROR	Target response error Occurs when detecting ERROR in m_hresp (DMA transaction). 0 No error 1 Error
6 ERRORINTRSTS_ ADMAERROR	DMA error This bit is set when the host controller detects errors during ADMA based data transfer. The state of the ADMA at an error occurrence is saved in the ADMA Error status register. 0 No error 1 Error
7 ERRORINTRSTS_ AUTOCMDERROR	Auto CMD error Auto CMD12 and Auto CMD23 use this error status. This bit is set when detecting that one of the bits D00-D04 in Auto CMD Error status register has changed from 0 to 1. In case of Auto CMD12, this bit is set to 1, not only when the errors in Auto CMD12 occur but also when Auto CMD12 is not executed due to the previous command error. 0 No error 1 Error
9 ERRORINTRSTS_ DATAENDBITERROR	Data end bit error Occurs when detecting 0 at the end bit position of read data which uses the DAT line or the end bit position of the CRC status. 0 No error 1 Error
10 ERRORINTRSTS_ DATA_CRCERROR	Data CRC error Occurs when detecting CRC error when transferring read data which uses the DAT line or when detecting the write CRC status having a value of other than "010". 0 No error 1 Error
11 ERRORINTRSTS_ DATATIMEOUTERROR	Data timeout error Occurs when detecting one of following timeout conditions: – Busy timeout for R1b, R5b type – Busy timeout after Write CRC status – Write CRC status timeout – Read data timeout 0 No error 1 Timeout
12 ERRORINTRSTS_ CMDINDEXERROR	Command index error Occurs if a command index error occurs in the command response. 0 No error 1 Error
13 ERRORINTRSTS_ CMDENDBITERROR	Command end bit error Occurs when detecting that the end bit of a command response is 0. 0 No error 1 End bit error generated

Table 1832. ERRORINTRSTS field descriptions (continued)

Field	Description
14 ERRORINTRSTS_ CMDCRCERROR	Command CRC error Command CRC error is generated in two cases. – If a response is returned and the command timeout error is set to 0, this bit is set to 1 when detecting a CRT error in the command response – The HC detects a CMD line conflict by monitoring the CMD line when a command is issued. If the HC drives the CMD line to 1 level, but detects 0 level on the CMD line at the next sdmmcctrl_clk edge, then the HC aborts the command (stop driving CMD line) and set this bit to 1. The command timeout error is also set to 1 to distinguish CMD line conflict. 0 No error 1 CRC error generated
15 ERRORINTRSTS_ CMDTIMEOUTERROR	Command timeout error Occurs only if the no response is returned within 64 sdmmcctrl_clk cycles from the end bit of the command. If the HC detects a CMD line conflict, in which case command CRC error is also be set. This bit is set without waiting for 64 sdmmcctrl_clk cycles because the command will be aborted by the HC. 0 No error 1 Timeout

Table 1833. Relation between command CRC error and command timeout error

Command CRC error	Command timeout error	Kinds of error
0	0	No error
0	1	Response timeout error
1	0	Response CRC error
1	1	CMD line conflicts

57.3.2.20 Normal interrupt status enable register (NORMALINTRSTSENA)

The following table describes the normal interrupt status enable register. Setting to 1 enables it.

Offset: 0x0034

Access: Read/Write

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	NORMALINTRSTS_ ENBLEREGBIT15	NORMALINTRSTS_ ENBLEREGBIT14	NORMALINTRSTS_ ENBLEREGBIT13	NORMALINTRSTS_ ENBLEREGBIT12	NORMALINTRSTS_ ENBLEREGBIT11	NORMALINTRSTS_ ENBLEREGBIT10	NORMALINTRSTS_ ENBLEREGBIT9	SDHCREGSET_ CARDINTSTSENA	SDHCREGSET_ CARDREMSTSENA	SDHCREGSET_ CARDINSSTSENA	NORMALINTRSTS_ ENBLEREGBIT5	NORMALINTRSTS_ ENBLEREGBIT4	NORMALINTRSTS_ ENBLEREGBIT3	NORMALINTRSTS_ ENBLEREGBIT2	NORMALINTRSTS_ ENBLEREGBIT1	NORMALINTRSTS_ ENBLEREGBIT0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1850. Normal interrupt status enable register (NORMALINTRSTSENA)

Table 1834. NORMALINTRSTSENA field descriptions

Field	Description
0 NORMALINTRSTS_ ENABLEREGBIT15	Fixed to 0 The HC controls error interrupts using the Error interrupt status enable register.
1 NORMALINTRSTS_ ENABLEREGBIT14	Boot terminate interrupt enable 0 Masked 1 Enabled Note: SoC does not support Execute In Place from eMMC.
2 NORMALINTRSTS_ ENABLEREGBIT13	Boot ack rcv enable 0 Masked 1 Enabled Note: SoC does not support Execute In Place from eMMC.
3 NORMALINTRSTS_ ENABLEREGBIT12	Re-tuning event status enable 0 Masked 1 Enabled Note: Re-tuning is not supported.
4 NORMALINTRSTS_ ENABLEREGBIT11	INT_C status enable If this bit is set to 0, the host controller clears the interrupt request to the system. The host driver may clear this bit before servicing the INT_C and may set this bit again after all interrupt requests to INT_C pin are cleared to prevent inadvertent interrupts. Note: INT_C is not supported.
5 NORMALINTRSTS_ ENABLEREGBIT10	INT_B status enable If this bit is set to 0, the host controller clears the interrupt request to the system. The host driver may clear this bit before servicing the INT_B and may set this bit again after all interrupt requests to INT_B pin are cleared to prevent inadvertent interrupts. Note: INT_B is not supported.
6 NORMALINTRSTS_ ENABLEREGBIT9	INT_A status enable If this bit is set to 0, the host controller clears the interrupt request to the system. The host driver may clear this bit before servicing the INT_A and may set this bit again after all interrupt requests to INT_A pin are cleared to prevent inadvertent interrupts. Note: INT_A is not supported.
7 SDHCREGSET_ CARDINTSTSENA	Card interrupt status enable If this bit is set to 0, the HC clears interrupt request to the system. The card interrupt detection is stopped when this bit is cleared and restarted when this bit is set to 1. The HD may clear the card interrupt status enable before servicing the card interrupt and may set this bit again after all interrupt requests from the card are cleared to prevent inadvertent interrupts. 0 Masked 1 Enabled
8 SDHCREGSET_ CARDREMSTSENA	Card removal status enable 0 Masked 1 Enabled

Table 1834. NORMALINTRSTSENA field descriptions (continued)

Field	Description
9 SDHCREGSET_ CARDINSSTSENA	Card insertion status enable 0 Masked 1 Enabled
10 NORMALINTRSTS_ ENABLEREGBIT5	Buffer read ready status enable 0 Masked 1 Enabled
11 NORMALINTRSTS_ ENABLEREGBIT4	Buffer write ready status enable 0 Masked 1 Enabled
12 NORMALINTRSTS_ ENABLEREGBIT3	DMA interrupt status enable 0 Masked 1 Enabled
13 NORMALINTRSTS_ ENABLEREGBIT2	Block gap event status enable 0 Masked 1 Enabled
14 NORMALINTRSTS_ ENABLEREGBIT1	Transfer complete status enable 0 Masked 1 Enabled
15 NORMALINTRSTS_ ENABLEREGBIT0	Command complete status enable 0 Masked 1 Enabled

57.3.2.21 Error interrupt status enable register (ERRORINTRSTSENA)

Setting to 1 enables interrupt status. The following table describes the Error interrupt status enable register.

Offset: 0x0036

Access: Read/Write

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W				ERRORINTRSTS_ ENABLEREGBIT12			ERRORINTRSTS_ ENABLEREGBIT9	ERRORINTRSTS_ ENABLEREGBIT8		ERRORINTRSTS_ ENABLEREGBIT6	ERRORINTRSTS_ ENABLEREGBIT5	ERRORINTRSTS_ ENABLEREGBIT4	ERRORINTRSTS_ ENABLEREGBIT3	ERRORINTRSTS_ ENABLEREGBIT2	ERRORINTRSTS_ ENABLEREGBIT1	ERRORINTRSTS_ ENABLEREGBIT0
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1851. Error interrupt status enable register (ERRORINTRSTSENA)

Table 1835. ERRORINTRSTSENA field descriptions

Field	Description
3 ERRORINTRSTS_ ENABLEREGBIT12	Target response error signal enable 0 Masked 1 Enabled
6 ERRORINTRSTS_ ENABLEREGBIT9	ADMA error status enable 0 Masked 1 Enabled
7 ERRORINTRSTS_ ENABLEREGBIT8	Auto CMD12 error status enable 0 Masked 1 Enabled
9 ERRORINTRSTS_ ENABLEREGBIT6	Data end bit error 0 Masked 1 Enabled
10 ERRORINTRSTS_ ENABLEREGBIT5	STA_EN: data CRC error status enable 0 Masked 1 Enabled
11 ERRORINTRSTS_ ENABLEREGBIT4	Data timeout error status enable 0 Masked 1 Enabled
12 ERRORINTRSTS_ ENABLEREGBIT3	Command index error status enable 0 Masked 1 Enabled
13 ERRORINTRSTS_ ENABLEREGBIT2	Command end bit error status enable 0 Masked 1 Enabled
14 ERRORINTRSTS_ ENABLEREGBIT1	Command CRC error status enable 0 Masked 1 Enabled
15 ERRORINTRSTS_ ENABLEREGBIT0	Command timeout error status enable 0 Masked 1 Enabled

57.3.2.22 Normal interrupt signal enable register (NORMALINTRSIGENA)

This register is used to select which interrupt status is indicated to the host system as the interrupt. These status bits all share the sample 1 bit interrupt line. Setting any of these bits to 1 enables interrupt generation.

Offset: 0x0038

Access: Read/Write

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	NORMALINTRSTS_	NORMALINTRSTS_	NORMALINTRSTS_	NORMALINTRSTS_	NORMALINTRSTS_	NORMALINTRSTS_	NORMALINTRSTS_	SDHCREGSET_	SDHCREGSET_	SDHCREGSET_	NORMALINTRSTS_	NORMALINTRSTS_	NORMALINTRSTS_	NORMALINTRSTS_	NORMALINTRSTS_	NORMALINTRSTS_
W	ENABLEREGBIT15	ENABLEREGBIT14	ENABLEREGBIT13	ENABLEREGBIT12	ENABLEREGBIT11	ENABLEREGBIT10	ENABLEREGBIT9	CARDINTSTSENA	CARDREMSTSENA	CARDINSSSTSENA	ENABLEREGBIT5	ENABLEREGBIT4	ENABLEREGBIT3	ENABLEREGBIT2	ENABLEREGBIT1	ENABLEREGBIT0
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1852. Normal interrupt signal enable register (NORMALINTRSIGENA)

Table 1836. NORMALINTRSIGENA field descriptions

Field	Description
0 NORMALINTRSTS_	Fixed to 0 The HD controls error interrupts using the Error interrupt signal enable register.
1 ENABLEREGBIT15	Boot terminate interrupt signal enable 0 Masked 1 Enabled Note: SoC does not support Execute In Place from eMMC.
2 ENABLEREGBIT14	Boot ack rcv signal enable 0 Masked 1 Enabled Note: SoC does not support Execute In Place from eMMC.
3 ENABLEREGBIT13	Re-tuning event signal enable 0 Masked 1 Enabled Note: Re-tuning is not supported.
4 ENABLEREGBIT12	INT_C signal enable 0 Masked 1 Enabled Note: INT_C is not supported.
5 ENABLEREGBIT11	INT_B signal enable 0 Masked 1 Enabled Note: INT_B is not supported.
6 ENABLEREGBIT10	INT_A signal enable 0 Masked 1 Enabled Note: INT_A is not supported.
7 ENABLEREGBIT9	Card interrupt signal enable 0 Masked 1 Enabled
SDHCREGSET_	
CARDINTSTSENA	

Table 1836. NORMALINTRSIGENA field descriptions (continued)

Field	Description
8 SDHCREGSET_ CARDREMSTSENA	Card removal signal enable 0 Masked 1 Enabled
9 SDHCREGSET_ CARDINSSTSENA	Card insertion signal enable 0 Masked 1 Enabled
10 NORMALINTRSTS_ ENABLEREGBIT5	Buffer read ready signal enable 0 Masked 1 Enabled
11 NORMALINTRSTS_ ENABLEREGBIT4	Buffer write ready signal enable 0 Masked 1 Enabled
12 NORMALINTRSTS_ ENABLEREGBIT3	DMA interrupt signal enable 0 Masked 1 Enabled
13 NORMALINTRSTS_ ENABLEREGBIT2	Block gap event signal enable 0 Masked 1 Enabled
14 NORMALINTRSTS_ ENABLEREGBIT1	Transfer complete signal enable 0 Masked 1 Enabled
15 NORMALINTRSTS_ ENABLEREGBIT0	Command complete signal enable 0 Masked 1 Enabled

57.3.2.23 Error interrupt signal enable register (ERRORINTRSIGENA)

This register is used to select which interrupt status is notified to the host system as the interrupt. These status bits all share the same 1 bit interrupt line. Setting any of these bits to 1 enables interrupt generation.

Offset: 0x003A

Access: Read/Write

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W				ERRORINTRSIG_ ENABLEREGBIT12		ERRORINTRSIG_ ENABLEREGBIT10	ERRORINTRSIG_ ENABLEREGBIT9	ERRORINTRSIG_ ENABLEREGBIT8	ERRORINTRSIG_ ENABLEREGBIT7	ERRORINTRSIG_ ENABLEREGBIT6	ERRORINTRSIG_ ENABLEREGBIT5	ERRORINTRSIG_ ENABLEREGBIT4	ERRORINTRSIG_ ENABLEREGBIT3	ERRORINTRSIG_ ENABLEREGBIT2	ERRORINTRSIG_ ENABLEREGBIT1	ERRORINTRSIG_ ENABLEREGBIT0
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1853. Error interrupt signal enable register (ERRORINTRSIGENA)

Table 1837. ERRORINTRSIGENA field descriptions

Field	Description
3 ERRORINTRSIG_ ENABLEREGBIT12	Target response error signal enable 0 Masked 1 Enabled
5 ERRORINTRSIG_ ENABLEREGBIT10	Tuning error signal enable 0 Masked 1 Enabled Note: Re-tuning is not supported.
6 ERRORINTRSIG_ ENABLEREGBIT9	ADMA error signal enable 0 Masked 1 Enabled
7 ERRORINTRSIG_ ENABLEREGBIT8	Auto CMD12 error signal enable 0 Masked 1 Enabled
8 ERRORINTRSIG_ ENABLEREGBIT7	Current limit error signal enable 0 Masked 1 Enabled
9 ERRORINTRSIG_ ENABLEREGBIT6	Data end bit error signal enable 0 Masked 1 Enabled
10 ERRORINTRSIG_ ENABLEREGBIT5	Data CRC error signal enable 0 Masked 1 Enabled
11 ERRORINTRSIG_ ENABLEREGBIT4	Data timeout error signal enable 0 Masked 1 Enabled
12 ERRORINTRSIG_ ENABLEREGBIT3	Command index error signal enable 0 Masked 1 Enabled
13 ERRORINTRSIG_ ENABLEREGBIT2	Command end bit error signal enable 0 Masked 1 Enabled
14 ERRORINTRSIG_ ENABLEREGBIT1	Command CRC error signal enable 0 Masked 1 Enabled
15 ERRORINTRSIG_ ENABLEREGBIT0	Command timeout error signal enable 0 Masked 1 Enabled

57.3.2.24 Auto CMD error status enable register (AUTOCMDERRSTS)

This register is used to indicate CMD12 response error of Auto CMD12 and CMD23 response error of Auto CMD23. The host driver can determine what kind of Auto CMD12/CMD23 errors occur by this register. Auto CMD23 errors are indicated in bit 04-01. This register is valid only when the Auto CMD error is set.

Note: This register is read only and initialized to zero on reset.

Offset: 0x003C

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	AUTOCMDERRSTS_ NEXTERROR	0	0	AUTOCMDERRSTS_ INDEXERROR	AUTOCMDERRSTS_ ENDBITERROR	AUTOCMDERRSTS_ CRCERROR	AUTOCMDERRSTS_ TIMEOUTERROR	AUTOCMDERRSTS_ NOTEXECERROR
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1854. Auto CMD error status enable register (AUTOCMDERRSTS)

Table 1838. AUTOCMDERRSTS field descriptions

Field	Description
8 AUTOCMDERRSTS_ NEXTERROR	Command not issued by Auto CMD12 error Setting this bit to 1 means CMD_wo_DAT is not executed due to an Auto CMD12 error (D04- D01) in this register. This bit is set to 0 when Auto CMD error is generated by Auto CMD23. 0 No error 1 Not issued
11 AUTOCMDERRSTS_ INDEXERROR	Auto CMD index error Occurs if the command index error occurs in response to a command. 0 No error 1 Error
12 AUTOCMDERRSTS_ ENDBITERROR	Auto CMD end bit error Occurs when detecting that the end bit of command response is 0. 0 No error 1 End bit error generated
13 AUTOCMDERRSTS_ CRCERROR	Auto CMD CRC error Occurs when detecting a CRC error in the command response. 0 No error 1 CRC error generated

Table 1838. AUTOCMDERRSTS field descriptions (continued)

Field	Description
14 AUTOCMDERRSTS_ TIMEOUTERROR	Auto CMD timeout error Occurs if the no response is returned within 64 sdmmcctrl_clk cycles from the end bit of the command. If this bit is set to 1, the other error status bits (D04 - D02) are meaningless. 0 No error 1 Timeout
15 AUTOCMDERRSTS_ NOTEXECERROR	Auto CMD12 not executed If memory multiple block data transfer is not started due to command error, this bit is not set because it is not necessary to issue Auto CMD12. Setting this bit to 1 means the HC cannot issue Auto CMD12 to stop memory multiple block transfer due to some error. If this bit is set to 1, other error status bits (D04 - D01) are meaningless. This bit is set to 0 when Auto CMD error is generated by Auto CMD23. 0 Executed 1 Not executed

Table 1839. Relation between auto CMD12 CRC error and auto CMD12 timeout error

Auto CMD12 CRC error	Auto CMD12 timeout error	Kinds of error
0	0	No error
0	1	Response timeout error
1	0	Response CRC error
1	1	CMD line conflict

The timing of changing Auto CMD12 error status can be classified in three scenarios:

- When the HC is going to issue Auto CMD12.
Set D00 to 1 if Auto CMD12 cannot be issued due to an error in the previous command.
Set D00 to 0 if Auto CMD12 is issued.
- At the end bit of Auto CMD12 response.
Check received responses by checking the error bits D01, D02, D03, D04.
Set to 1 if error is detected.
Set to 0 if error is not detected.
- Before reading the Auto CMD12 error status bit D07.
Set D07 to 1 if there is a command cannot be issued.
Set D07 to 0 if there is no command to issue.

Timing of generating the Auto CMD12 error and writing to the Command register are asynchronous. Then D07 is sampled when driver never writing to the Command register. So just before reading the Auto CMD12 Error status register is good timing to set the D07 status bit.

57.3.2.25 Host control 2 register (HOSTCONTROL2)

The following table describes the Host control 2 register.

Offset: 0x003E

Access: Read/Write

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	HOSTCTRL2_	HOSTCTRL2_	0	0	0	0	0	0	HOSTCTRL2_	HOSTCTRL2_	0		HOSTCTRL2_			
W	PRESETVALUEENABLE	ASYNCHINTREENABLE							SAMPLINGCLKSELECT	EXECUTETUNING	DRIVERSTRENGTH		1P8VSIGNALINGENA		UHSMODESELECT	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1855. Host control 2 register (HOSTCONTROL2)

Table 1840. HOSTCONTROL2 field descriptions

Field	Description
0 HOSTCTRL2_ PRESETVALUEENABLE	Preset value enable As the operating sdmmccard_clk frequency and I/O driver strength depend on the host system implementation, it is difficult to determine these parameters in the standard host driver. When preset value enable is set to automatic. This bit enables the functions defined in the Preset value registers. 1 Automatic selection by preset value are enabled 0 sdmmccard_clk and driver strength are controlled by host driver If this bit is set to 0, sdmmccard_clk frequency select and driver strength select in Host control 2 register are set by host driver. If this bit is set to 1, sdmmccard_clk frequency Select and driver strength select in Host control 2 register are set by host controller as specified in the Preset value registers.
1 HOSTCTRL2_ ASYNCHINTREENABLE	Asynchronous interrupt enable 1 Enabled 0 Disabled
8 HOSTCTRL2_ SAMPLINGCLKSELECT	Sampling clock select 1 Tuned clock is used to sample data 0 Fixed clock is used to sample data Note: Tuning is not supported. This bit should always set to 0.
9 HOSTCTRL2_ EXECUTETUNING	Execute tuning 1 Execute tuning 0 Not tuned or tuning completed Note: Tuning is not supported. This bit should always set to 0.
10:11 HOSTCTRL2_ DRIVERSTRENGTH	Driver strength select In 3.3V signaling, this field is not effective. Note: This field is not effective as only 3.3V signaling is supported.

Table 1840. HOSTCONTROL2 field descriptions (continued)

Field	Description
12 HOSTCTRL2_1P8VSIGNALINGENA	1.8V signaling enable This bit controls voltage regulator for I/O cell. 3.3V is supplied to the card regardless of signaling voltage. Note: This field is not effective as only 3.3V signaling is supported.
13:15 HOSTCTRL2_UHSMODESELECT	This field is used to select DDR mode of eMMC. Set it to 100b to enable High speed DDR mode.

57.3.2.26 Capabilities[31:0] register (CAPABILITIES31_0)

This register provides the HD with information specific to the HC implementation. The HC may implement these values as fixed or loaded from Flash memory during power on initialization.

Offset: 0x0040

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15							
R	CORECFG_SLOTTYPE		CORECFG_ASYNCINTRSUPPORT		0	0	0	0	CORECFG_3P3VOLTSUPPORT		CORECFG_SUSPENDRESUME		CORECFG_SDMA2SUPPORT		CORECFG_HIGHSPEEDSUPPORT		0	CORECFG_ADMA2SUPPORT		CORECFG_8BITSUPPORT		CORECFG_MAXBLKLENGTH	
W																							
Reset	0	0	1	0	0	0	0	1	1	1	1	0	0	1	0	0		0	0	1	0	0	

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	CORECFG_BASECLKFREQ								CORECFG_TIMEOUTCLKUNIT		0	CORECFG_TIMEOUTCLKFREQ				
W																
Reset	0	0	1	1	0	0	1	0	1	0	0	0	0	0	0	1

Figure 1856. Capabilities[31:0] register (CAPABILITIES31_0)

Table 1841. CAPABILITIES31_0 field descriptions

Field	Description
0:1 CORECFG_ SLOTTYPE	Slot type support This field indicates usage of a slot by a specific host system. 00 Removable card slot 01 Embedded slot for one device 10 Shared bus slot 11 Reserved
2 CORECFG_ ASYNCHINTRSUPPORT	CORECFG_ASYNCHINTRSUPPORT Enable asynchronous interrupt support 0 Core does not support monitoring of asynchronous interrupt 1 Core supports monitoring of asynchronous interrupt
7 CORECFG_ 3P3VOLTSUPPORT	Voltage support 3.3V This bit indicates whether the HC supports 3.3V. 0 3.3V Not supported 1 3.3V Supported Note: 3.3V is supported
8 CORECFG_ SUSPENDRESUME	CORECFG_SUSPENDRESUME SDMMC host controller supports Suspend/Resume functionality. 0 Not supported 1 Supported
9 CORECFG_ SDMASUPPORT	SDMA support This bit indicates whether the HC is capable of using DMA to transfer data between system memory and the HC directly. 0 SDMA Not supported 1 SDMA Supported
10 CORECFG_ HIGHSPEEDSUPPORT	High speed support This bit indicates whether the HC and the host system support High speed mode and they can supply sdmmc_card_clk clock frequency from 20 MHz to 50 MHz. 0 High speed not supported 1 High speed supported
12 CORECFG_ ADMA2SUPPORT	ADMA2 support 0 ADMA2 not supported 1 ADMA2 supported Note: ADMA2 configuration is selected on programming of CORE_CONFIG.corecfg_adma2support bit.
13 CORECFG_ 8BITSUPPORT	Extended media bus support This bit indicates whether the host controller is capable of using 8-bit bus width mode. 0 Extended media bus not supported 1 Extended media bus supported

Table 1841. CAPABILITIES31_0 field descriptions (continued)

Field	Description
14:15 CORECFG_ MAXBLKLENGTH	Max block length This value indicates the maximum block size that the HD can read and write to the buffer in the HC. The buffer transfers this block size without wait cycles. Three sizes can be defined as indicated below. 00 512 bytes 01 1024 bytes 10 2048 bytes 11 4096 bytes Note: max 512 B block size is supported.
16:23 CORECFG_ BASECLKFREQ	Base clock frequency of SDMMC_CLK It's value is 8'b00110010 (50 MHz). 50 MHz is shown as reset value, but sdmmc_clk freq is 100 MHz. 50 MHz clk for SDMMC card and controller should be generated by software by dividing sdmmc_clk from 2. Note: Base clock frequency configuration is programmed in CORE_CONFIG.baseclkfreq_sdmmc_clk field.
24 CORECFG_ TIMEOUTCLKUNIT	Timeout clock unit This bit shows the unit of base clock frequency used to detect data timeout error. 0 KHz 1 MHz
26:31 CORECFG_ TIMEOUTCLKFREQ	Timeout clock frequency This bit shows the base clock frequency used to detect data timeout error. It's value is 6'b000001. Because timeout clock unit is 1 MHz, so timeout clock frequency is 1 MHz.

57.3.2.27 Capabilities[63:32] register (CAPABILITIES63_32)

This register provides the HD with information specific to the HC implementation. The HC may implement these values as fixed or loaded from Flash memory during power on initialization.

Offset: 0x0044

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	CORECFG_SPIBLKMODE	CORECFG_SPISUPPORT	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	CORECFG_RETUNINGMODES		0	0	0	0	0	0	0	0	0	0	0	CORECFG_DDRMODESUPPORT	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

Figure 1857. Capabilities[63:32] register (CAPABILITIES63_32)

Table 1842. CAPABILITIES63_32 field descriptions

Field	Description
6 CORECFG_SPIBLKMODE	SPI block mode This field indicates whether SPI block mode is supported or not. 0 Not supported 1 Supported
7 CORECFG_SPISUPPORT	SPI mode This field indicates whether SPI mode is supported or not. 0 Not supported 1 Supported
16:17 CORECFG_RETUNINGMODES	Re-tuning modes This field defines the re-tuning capability of a host controller. Note: Re-tuning is not supported. This field is set to 2'b00.
29 CORECFG_DDRMODESUPPORT	DDR mode support 1 DDR mode is supported. 0 DDR mode is not supported.

57.3.2.28 Maximum current capabilities[31:0] register (MAXCURRENTCAP31_0)

This register describes the maximum current capabilities.

Offset: 0x0048

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	0	0	0	0	0	0	0	0	CORECFG_MAXCURRENT3P3V							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1858. Maximum current capabilities[31:0] register (MAXCURRENTCAP31_0)

Table 1843. MAXCURRENTCAP31_0 field descriptions

Field	Description
24:31 CORECFG_ MAXCURRENT3P3V	Maximum current for 3.3V

57.3.2.29 Maximum current capabilities[63:32] register (MAXCURRENTCAP63_32)

This register describes the maximum current capabilities.

Offset: 0x004C

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1859. Maximum current capabilities[63:32] register (MAXCURRENTCAP63_32)

Table 1844. MAXCURRENTCAP63_32 field descriptions

Field	Description
0:31	Reserved

[Table 1845](#) describes the maximum current value.

Table 1845. Maximum current value definition

Register value	Current value
0	Get information via another method
1	4 mA
2	8 mA
3	12 mA
...	
254	1016 mA
255	1020 mA

Note: Maximum current values are required to be programmed in `CORE_CONFIG.maxcurrent_3p3v` register field.

57.3.2.30 Preset value n register (PRESETVALUE_n)

Preset value for registers 0 to 7 is all 0. It is because only 3.3V signaling is supported and clock generator is not supported.

Offset: $0x0060 + n \times 0x2$ ($n=0$ to 7)

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1860. Preset value n register (PRESETVALUE_n)Table 1846. PRESETVALUE_n field descriptions

Field	Description
0:15	Reserved. Refer to Table 1847 for details.

[Table 1847](#) shows the offset of preset values per card or device.

Table 1847. Preset value register definition

Register name	Offset	Preset value register	Signal
reg_presetvalue_0	0x0060	Preset value for Initialization	3.3 V
reg_presetvalue_1	0x0062	Preset value for Default speed	3.3 V
reg_presetvalue_2	0x0064	Preset value for High speed	3.3 V
reg_presetvalue_3	0x0066	Reserved	
reg_presetvalue_4	0x0068	Reserved	
reg_presetvalue_5	0x006A	Reserved	

Table 1847. Preset value register definition (continued)

Register name	Offset	Preset value register	Signal
reg_presetvalue_6	0x006C	Reserved	
reg_presetvalue_7	0x006E	Reserved	

57.3.2.31 Slot interrupt status register (SLOTINTRSTS)

The following table describes the slot interrupt status register.

Offset: 0x00FC

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	SDHCHOSTIF SLOTINTRSTS
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1861. Slot interrupt status register (SLOTINTRSTS)

Table 1848. SLOTINTRSTS field descriptions

Field	Description
15 SDHCHOSTIF_ SLOTINTRSTS	Interrupt signal for slot#0 This status bit indicates the OR of interrupt signal and wakeup signal for slot. Note: Wakeup interrupt is not supported. Bit[0] is showing status of interrupt signal only.

57.3.2.32 Host controller version register (HOSTCONTROLLERVER)

The following table describes the Host controller version register.

Offset: 0x00FE

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	SDHC_VENVERNUM								SPECIFICATIONVERSIONNUMBER							
W																
Reset	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0

Figure 1862. Host controller version register (HOSTCONTROLLERVER)

Table 1849. HOSTCONTROLLERVER field descriptions

Field	Description
0:7 SDHC_VENVERNUM	Vendor version number The vendor version number is set to 0x10 (1.0)
8:15 SPECIFICATION VERSIONNUMBER	Specification version number The host controller version number is set to 0x02 (Host specification version 3.00).

57.3.2.33 Force event for auto CMD error status register (FORCEEVENTFORAUTOCMDERRSTS)

The Force event register is not a physically implemented register. Rather, it is an address at which the Auto CMD12 Error status register can be written:

- Writing 1: set each bit of the Auto CMD12 Error status register
- Writing 0: no effect

Offset: 0x0050

Access: Write only

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0		0	0					
W									FORCECMDNOTISSUED BYAUTOCMD12ERR			FORCEAUTOCMD INDEXERR	FORCEAUTOCMD ENDBITERR	FORCEAUTOCMD CRCERR	FORCEAUTOCMD TIMEOUTERR	FORCEAUTOCMD NOTEXEC
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1863. Force event for auto CMD error status register (FORCEEVENTFORAUTOCMDERRSTS)

Table 1850. FORCEEVENTFORAUTOCMDERRSTS field descriptions

Field	Description
8 FORCECMDNOTISSUED BYAUTOCMD12ERR	Force event for command not issued by Auto CMD12 error 1 Interrupt is generated 0 No interrupt
11 FORCEAUTOCMD INDEXERR	Force event for Auto CMD index error 1 Interrupt is generated 0 No interrupt
12 FORCEAUTOCMD ENDBITERR	Force event for Auto CMD index error 1 Interrupt is generated 0 No interrupt
13 FORCEAUTOCMD CRCERR	Force event for Auto CMD CRC error 1 Interrupt is generated 0 No interrupt

Table 1850. FORCEEVENTFORAUTOCMDERRSTS field descriptions (continued)

Field	Description
14 FORCEAUTOCMD TIMEOUTERR	Force event for Auto CMD timeout error 1 Interrupt is generated 0 No interrupt
15 FORCEAUTOCMD NOTEXEC	Force event for Auto CMD12 NOT executed 1 Interrupt is generated 0 No interrupt

57.3.2.34 Force event for error interrupt status register (FORCEEVENTFORERRINTSTS)

The Force event register is not a physically implemented register. Rather, it is an address at which the Error interrupt status register can be written. The effect of a write to this address will be reflected in the Error interrupt status register if the corresponding bit of the Error interrupt status enable register is set:

- Writing 1: set each bit of the Error interrupt status register
- Writing 0: no effect

Offset: 0x0052

Access: Write only

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0										
W							FORCE ADMAERR	FORCE AUTOCMDERR	FORCE CURRLIMERR	FORCE DATENDBITERR	FORCE DATCRCERR	FORCE DATTIMEOUTERR	FORCE CMDINDEXERR	FORCE CMDENDBITERR	FORCE CMDCRCERR	FORCE CMDTIMEOUTERR
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1864. Force event for error interrupt status register (FORCEEVENTFORERRINTSTS)**Table 1851. FORCEEVENTFORERRINTSTS field descriptions**

Field	Description
6 FORCEADMAERR	Force event for ADMA error 1 Interrupt is generated 0 No interrupt
7 FORCEAUTOCMDERR	Force event for Auto CMD error 1 Interrupt is generated 0 No interrupt
8 FORCECURRLIMERR	Force event for current limit error 1 Interrupt is generated 0 No interrupt
9 FORCEDATENDBITERR	Force event for data end bit error 1 Interrupt is generated 0 No interrupt

Table 1851. FORCEEVENTFORERRINTSTS field descriptions (continued)

Field	Description
10 FORCEDATCRCERR	Force event for data CRC error 1 1 Interrupt is generated 0 No interrupt
11 FORCEDATTIMEOUTERR	Force event for data timeout error 1 1 Interrupt is generated 0 No interrupt
12 FORCECMDINDEXERR	Force event for command index error 1 1 Interrupt is generated 0 No interrupt
13 FORCECMDENDBITERR	Force event for command end bit error 1 Interrupt is generated 0 No interrupt
14 FORCECMDCRCERR	Force event for command CRC error 1 Interrupt is generated 0 No interrupt
15 FORCECMDTIMEOUTERR	Force event for command timeout error 1 Interrupt is generated 0 No interrupt

57.3.2.35 ADMA Error status register (ADMAERRSTS)

When ADMA error interrupt occurs, the ADMA error states field in this register holds the ADMA state and the ADMA System address register holds the address around the error descriptor. The following table describes the ADMA Error interrupt register.

Offset: 0x0054

Access: Read

	0	1	2	3	4	5	6	7
R	0	0	0	0	0	ADMAERRSTS_	ADMAERRSTS_	
						ADMAENMISMATCHERR	ADMAERRORSTATE	
W								
Reset	0	0	0	0	0	0	0	0

Figure 1865. ADMA Error status register (ADMAERRSTS)

Table 1852. ADMAERRSTS field descriptions

Field	Description
5 ADMAERRSTS_ ADMALENMISMATCHERR	ADMA length mismatch error This error occurs in the following 2 cases. While block count enable being set, the total data length specified by the descriptor table is different from that specified by the block count and block length. Total data length can not be divided by the block length. 1 Error 0 No error
6:7 ADMAERRSTS_ ADMAERRORSTATE	ADMA error state This field indicates the state of ADMA when error is occurred during ADMA data transfer. This field never indicates "10" because ADMA never stops in this state. D01–D00: ADMA error state when error occurred. Contents of SYS_SDR register: 00 ST_STOP (Stop DMA) points to next of the error descriptor 01 ST_FDS (Fetch Descriptor) points to the error descriptor 10 Never set this state (Not used) 11 ST_TFR (Transfer Data) points to the next of the error descriptor

57.3.2.36 ADMA System address n register (ADMASYSADDRn)

The following table describes the ADMA System address register.

Offset: 0x0058 + n*0x2 (n=0 to 3)

Access: Read/Write

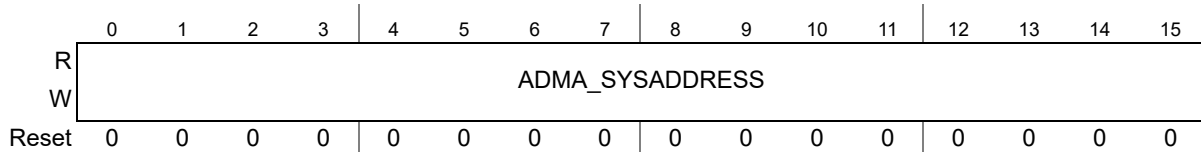


Figure 1866. ADMA System address n register (ADMASYSADDRn)

Table 1853. ADMASYSADDRn field descriptions

Field	Description
0:15 ADMA_SYSADDRESS	ADMA system address This register holds byte address of executing command of the descriptor table. 32-bit Address descriptor uses lower 32-bit of this register. At the start of ADMA, the host driver set start address of the descriptor table. The ADMA increments this register address, which points to next line, when every fetching a descriptor line. When the ADMA error interrupt is generated, this register holds valid descriptor address depending on the ADMA state. The host driver programs descriptor table on 32-bit boundary and set 32-bit boundary address to this register. ADMA2 ignores lower 2-bit of this register and assumes it to be 00b. 32-bit Address ADMA register value 32-bit system address XXXXXXXX 0x00000000 0x00000000 XXXXXXXX 0x00000004 0x00000004 ... XXXXXXXX 0xFFFFFFFFC 0xFFFFFFFFC

57.3.2.37 Boot timeout control register (BOOTTIMEOUTCNT)

The following table describes the boot timeout control register.

Offset: 0x0070												Access: Read/Write				
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	BOOT_TIMEOUTCNT															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	BOOT_TIMEOUTCNT															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1867. Boot timeout control register (BOOTTIMEOUTCNT)

Table 1854. BOOTTIMEOUTCNT field descriptions

Field	Description
0:31 BOOT_TIMEOUTCNT	Boot data timeout counter value This value determines the interval by which DAT line timeouts are detected during boot operation for eMMC card. The value is in number of sdmmcclk clock. Note: SoC does not support Execute In Place from eMMC.

57.3.2.38 Core configuration register (CORE_CONFIG)

The register provides field to configure SDMMC protocol clock frequency (SDMMC_CLK), ADMA2 selection and maximum IO current value.

Offset: 0x0208

Access: Read/Write

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	MAXCURRENT_3P3V								0	0	0	0	CORECFG_SLOTTYPE	CORECFG_ASYNCINTRSUPPORT	CORECFG_ADMA2SUPPORT	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	0	0	0	0	0	0	0	0	BASECLKFREQ_SDMMC_CLK							
W																
Reset	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	0

Figure 1868. Core configuration register (CORE_CONFIG)

Table 1855. CORE_CONFIG field descriptions

Field	Description
0:7 MAXCURRENT_3P3V	Maximum current for 3.3V. Value depends on IO drive strength.
12:13 CORECFG_SLOTTYPE	Select slot type 00 Removable card slot 01 Embedded slot for one device 10 Shared bus slot 11 Reserved
14 CORECFG_ASYNCHINTRSUPPORT	Enable asynchronous interrupt support 0 Core does not support monitoring of asynchronous interrupt 1 Core supports monitoring of asynchronous interrupt
15 CORECFG_ADMA2SUPPORT	Selection for ADMA2 support
24:31 BASECLKFREQ_SDMMC_CLK	Base clock frequency for SDMMC clock. This is the frequency of the sdmmc_clk. 8'h32 = 50 MHz

57.3.2.39 Feedback clock selection register (FB_CLK_SEL)

The register is used to program the feedback clock source.

Offset: 0x0210

Access: Read/Write

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	FEEDBACKCLK_SEL	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1869. Feedback clock selection register (FB_CLK_SEL)

Table 1856. FB_CLK_SEL field descriptions

Field	Description
30:31 FEEDBACKCLK_SEL	Selects feedback clock from the sources of loopback. 00 Loopback from output of SDMMC HC IP (delayed SDMMC Tx clock is provided as loopback clock) 01 OPEN 10 OPEN 11 Loopback from external SDMMC device pin connected on SDMMC_FDBCK_CLK IO PAD

57.3.2.40 Debug status 1 register (DBG_STA1)

This register provides 16bit wide DMA Control debug bus for software debugging.

Offset: 0x0220Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	DMADEBUGBUS															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1870. Debug status 1 register (DBG_STA1)

Table 1857. DBG_STA1 field descriptions

Field	Description
16:31 DMADEBUGBUS	DMA_CTRL debug bus hostintf_blocknextcmd - Command Complete indication from CMDCTRL module is set, the next command indication should be blocked hostintf_abortcmdmode - When the Abort command is issued, the flag is set hostintf_rdxferactive - This is set from the time the command Last Bit is issued or Block Gap Continue to the time the last block of data is sent to host hostintf_enddataxfer - It is set when the end Data Transfer Complete Indications from the PIO/SDMA and ADMA2 state machines hostintf_stopatblkgap - For write transfer, it is set from the PIO/SDMA and ADMA2 state machines. For read transfer, RegSet is used to generate Stop at BlkGap hostwrdat_state - The WriteData state machine transfers the data to host interface hostrddat_state - The ReadData state machine receives the data from host interface hosttrans_state[1:0] - Host transfer state machine adma2_state[3:0] - ADMA2 state machine piosdma_state[2:0] - PIO/SDMA state machine

57.3.2.41 Debug status 2 register (DBG_STA2)

This register provides 16bit wide CMD Control debug bus for software debugging.

Note: There is a delay from generating the status signals inside the SDMMC core to update of the related status register bits due to clock domain crossing.

Offset: 0x0224

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	CMDDEBUGBUS															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1871. Debug status 2 register (DBG_STA2)

Table 1858. DBG_STA2 field descriptions

Field	Description
16:31 CMDDEBUGBUS	CMD_CTRL debug bus cmdfsm_cmdrespstatus [3:0] - Command response status sdhcregset_bootena_sdmmcclk - Double synchronization of sdhcregset_bootena signal sdhcregset_cmdexecute_sdmmcclk - Double synchronization of sdhcregset_cmdexecute signal cmdfsm_autocmd23 - It is set when AutoCMD 23 Qualified with CMD_Data Present cmdfsm_autocmd12 - Auto CMD12 indication cmdfsm_cmdissued - CMD issued signal cmdfsm_cmdcomplete - Command complete bit cmdfsm_cmdena - CMD output enable cmdfsm_cmdout - CMD output cmdfsm_state[3:0] - Command state machine

57.3.2.42 Debug status 3 register (DBG_STA3)

This register provides 16bit wide TXD module control debug bus for software debugging.

Note: There is a delay from generating the status signals inside the SDMMC HC core to update of the related status register bits due to clock domain crossing.

Offset: 0x0228

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	TXDDEBUGBUS															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1872. Debug status 3 register (DBG_STA3)

Table 1859. DBG_STA3 field descriptions

Field	Description
16:31 TXDDEBUGBUS	TXD_CTRL debug bus txdfsm_enatimeoutchk - Enable data timeout check txdfsm_stopsdmmccardclk - Stop SDMMC card clock txdfsm_xmitstatus[2:0] - Transmit status {ENDBIT err, CRC error, timeout error} txdfsm_xmitstsvld - Transmit status valid indication txdfsm_rcvcrcts - Receive CRC status indication to RXCRC state machine txdfsm_readeob - Read EOB txdfsm_readbuffer - Read buffer txdfsm_sdmmcdataena - SDMMC data enable txdfsm_wrxferactive - Wr transfer active indication txdfsm_datelineactive - Data line active indication txdfsm_state[3:0] - Transmit data state machine

57.3.2.43 Debug status 4 register (DBG_STA4)

This register provides 16bit wide RXD module control debug bus for software debugging.

Note: There is a delay from generating the status signals inside the SDMMC HC core to update of the related status register bits due to clock domain crossing.

Offset: 0x022C

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	0	0	0	0	0	0	0	0	RXDDEBUGBUS0							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1873. Debug status 4 register (DBG_STA4)

Table 1860. DBG_STA4 field descriptions

Field	Description
24:31 RXDDEBUGBUS0	RXD_CTRL debug bus rxctrl_rcvstatus[2:0] - Receive status {EndBit Err, CRC Error, timeout error} rxctrl_rcvstsvld - Receive status valid rxctrl_stopafterblk - Stop receiving after this block (Stop at block gap) rxctrl_stopspcardclk2 - Stop SDMMC clock indication (Stop at block gap) rxctrl_stopspcardclk1 - Stop SDMMC clock indication (Normal operation, Buffer full) rxctrl_rcvdata - Receive data indication

57.3.2.44 Debug status 5 register (DBG_STA5)

This register provides 16bit wide RXD module control debug bus on feedback clock for software debugging.

Note: There is a delay from generating the status signals inside the SDMMC HC core to update of the related status register bits due to clock domain crossing.

Offset: 0x0230

Access: Read

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	0	0	0	0	0	0	0	0	RXDDEBUGBUS1							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1874. Debug status 5 register (DBG_STA5)

Table 1861. DBG_STA5 field descriptions

Field	Description
24:31 RXDDEBUGBUS1	RXD_CTRL debug bus (RX CLK) rxdfsm_reachingeob - Reaching End of block indication rxdfsm_okstopclk - OK to Stop the clock (End of block flow control) rxdfsm_dataeob - Data Byte/Word is End of block indication rxdfsm_datawrite - Data Byte/Word write rxdfsm_wtforblk - Wait for block indication rxdfsm_state [2:0] - Receive data state machine

57.4 Driver flow sequence

This section provides the details about various data transfer protocols for 3MCR host controller.

Note: *FB_CLK_SEL.feedbackClk_sel[1:0] register field should be programmed for selection of feedback clock. For more detail refer to [Section 57.2.3: Clocks](#).*

57.4.1 Non-DMA transaction

The sequence for Not Using DMA [non - DMA] is shown in [Figure 1875](#) and [Table 1862](#).

Figure 1875. Data transfer using DAT line sequence (not using DMA)

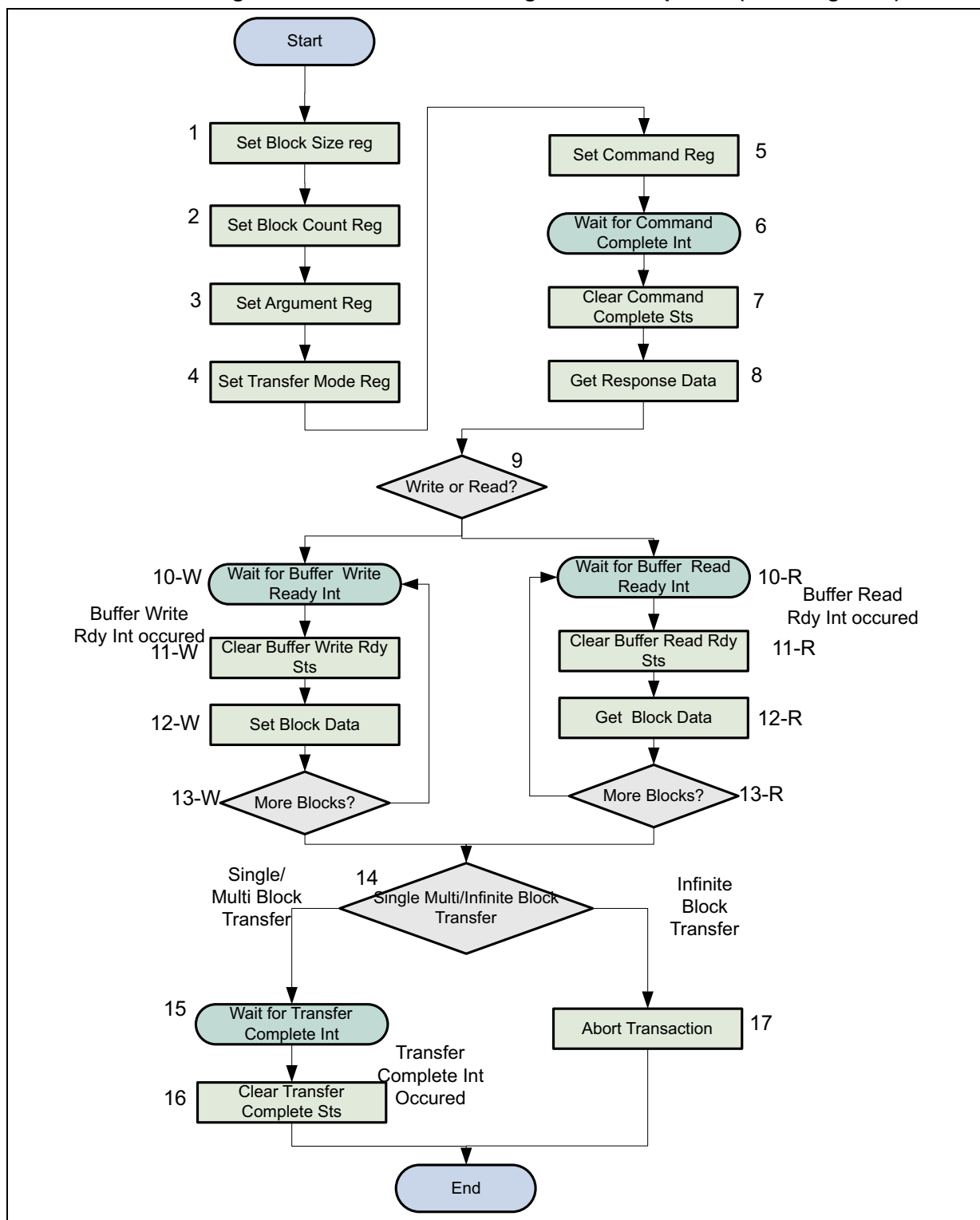


Table 1862. Non-DMA transaction

Step	Description
1	Set the value corresponding to the executed data byte length of one block to Block size register.
2	Set the value corresponding to the executed data block count to Block count register.
3	Set the value corresponding to the issued command to Argument register.
4	Set the value to Multi / Single Block Select and Block Count Enable. Set the value corresponding to the issued command to Data transfer direction, Auto CMD12 Enable and DMA Enable.
5	Set the value to Multi / Single Block Select and Block Count Enable. Set the value corresponding to the issued command to Data transfer direction, Auto CMD12 Enable and DMA Enable.
6	Wait for the Command Complete Interrupt
7	Write 1 to the Command Complete in the Normal interrupt status register for clearing this bit.
8	Read Response register and get necessary information in accordance with the issued command.
9	In the case where this sequence is for write to a card, go to step (10-W). In case of read from a card, go to step (10-R).
10-W	Wait for Buffer Write Ready interrupt. Non DMA Write transfer On receiving the Buffer Write Ready interrupt the system processor will act as a master and start transferring the data via Buffer data port register (FIFO_1). Transmitter starts sending the data in SDMMC bus when a block of data is ready in FIFO_1. While transmitting the data in SDMMC bus the Buffer Write Ready interrupt is sent to the system processor for the second block of data. The system processor will act as a master and start sending the second block of data via Buffer data port register to FIFO_2. Buffer Write Ready interrupt will be asserted only when a FIFO is empty to receive a block of data.
11-W	Write 1 to the Buffer Write Ready in the Normal interrupt status register for clearing this bit.
12-W	Write block data (in according to the number of bytes specified at the step (1)) to Buffer data port register.
13-W	Repeat until all blocks are sent and then go to step (14).
10-R	Non DMA Read transfer Buffer Read Ready interrupt is asserted whenever a block of data is ready in one of the FIFOs. On receiving the Buffer Read Ready interrupt the system processor will act as a master and start reading the data via Buffer data port register (FIFO_1). Receiver start reading the data from SDMMC bus only when a FIFO is empty to receive a block of data. When both the FIFOs are full the host controller will stop the data coming from the card through Read Wait mechanism (if card supports Read Wait) or through clock stopping. Wait for Buffer Read Ready interrupt
11-R	Write 1 to the Buffer Read Ready in the Normal interrupt status register for clearing this bit.
12-R	Read block data (in according to the number of bytes specified at the step (1)) from the Buffer data port register

Table 1862. Non-DMA transaction (continued)

Step	Description
13-R	Repeat until all blocks are received and then go to step (14).
14	If this sequence is for Single or Multiple block transfer, go to step (15). In case of Infinite block transfer, go to step (17).
15	Wait for Transfer Complete Interrupt.
16	Write 1 to the Transfer Complete in the Normal interrupt status register for clearing this bit.
17	Perform the sequence for Abort transaction.

Note: Step (1) and (2) can be executed at the same time. Step (4) and (5) can be executed at the same time.

57.4.2 DMA transaction

The burst types like 8-beat incrementing burst or 4-beat incrementing burst or single transfer is used to transfer or receive the data from the system memory mainly to avoid the hold of the Host/System bus by the master for a longer time.

The sequence for using DMA is shown in [Figure 1876](#) and [Table 1863](#).

Note: For data read/write using SDMA, swapping of bytes is not required. Swapping of bytes should be disabled using `EMMC_MSTR_EN_BYTE_SWAP` (bit 27) of PCM REG3. This bit should be reset to 0. Reset value of this bit is 1. SDMMC driver should reset it.

Figure 1876. Data transfer using DAT line sequence (using DMA)

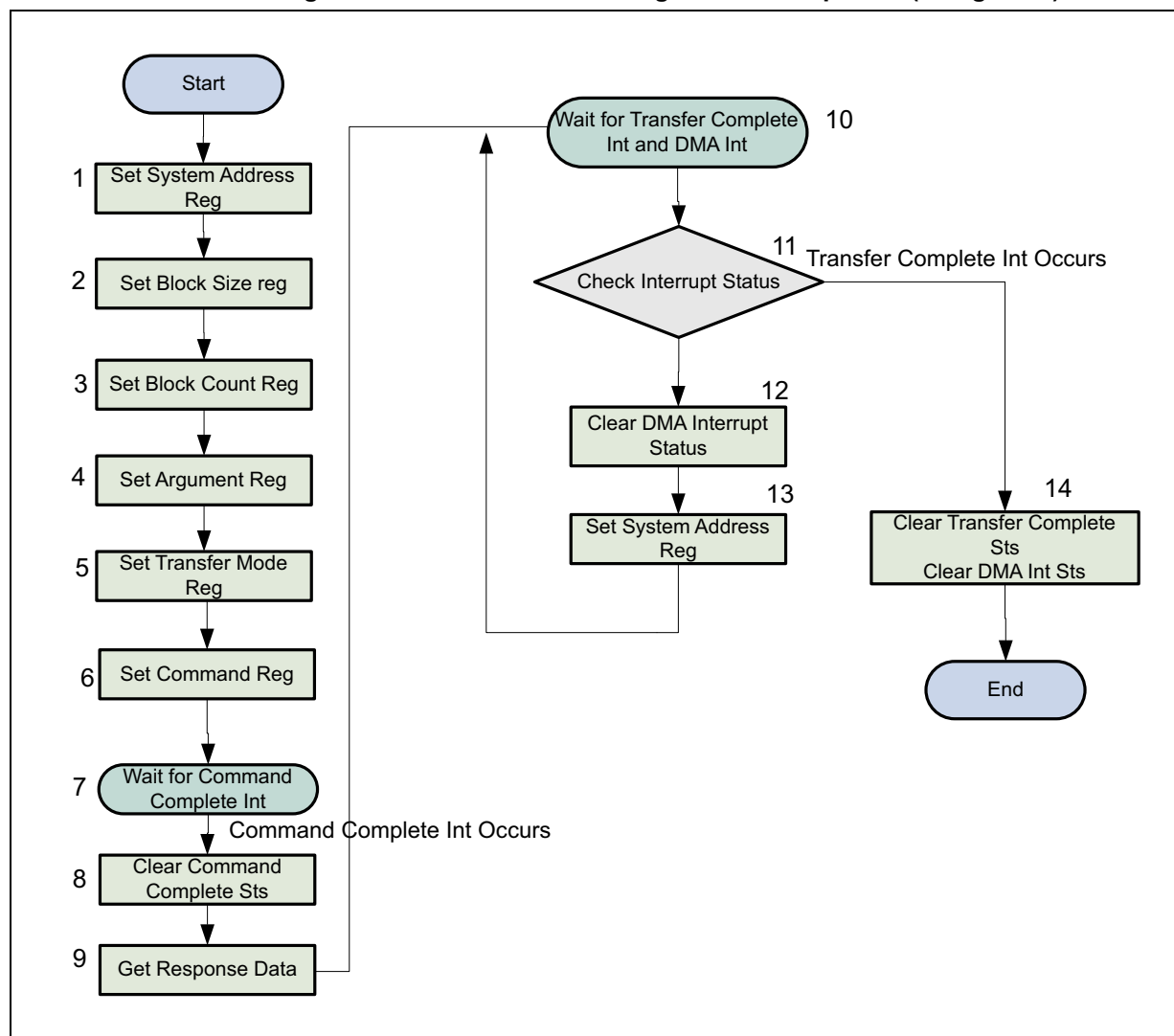


Table 1863. DMA transaction

Step	Description
1	Set the system address for DMA in the System address register.
2	Set the value corresponding to the executed data byte length of one block in the Block size register.
3	Set the value corresponding to the executed data block count in the Block count register.
4	Set the value corresponding to the issued command to Argument register.
5	Set the value to Multi / Single block select and Block count enable. Set the value corresponding to the issued command to Data transfer direction, Auto CMD12 Enable and DMA Enable.
6	Set the value corresponding to the issued command to Command register.
7	Wait for the Command Complete Interrupt.

Table 1863. DMA transaction (continued)

Step	Description
8	Write 1 to the Command Complete in the Normal interrupt status register for clearing this bit.
9	Read Response register and get necessary information in accordance with the issued command. DMA read transfer On receiving the response end bit from the card for the write command (data flowing from host to card) the host controller will act as the master and request the System/Host bus. After receiving the grant the host controller will start reading a block of data from the system memory and fills the first FIFO. Whenever a block of data is ready the transmitter will start sending the data in SDMMC bus. While transmitting the data in SDMMC bus the host controller requests the bus to fill the second block in second FIFO. Ping-pong FIFOs are used to increase the throughput. Similarly the host controller reads a block of data from the system memory whenever a FIFO is empty. This will continue till all the blocks are read from the system memory. Transfer Complete Interrupt will be set only after transferring all the blocks of data to the card. DMA write transfer The block of data received from the card (data flowing from card to host) is stored in first half of the FIFO. Whenever a block of data is ready the host controller will act as the master and request the System/Host bus. After receiving the grant the host controller will start writing a block of data into the system memory from the first FIFO. While transmitting the data into system memory the host controller will receive the second block of data and store in second FIFO. Similarly the host controller write a block of data into the system memory whenever data is ready. This will continue till all the blocks are transferred to the system memory. Transfer Complete Interrupt will be set only after transferring all the blocks of data to the system memory. Note: Host controller will receive a block of data from the card only when it has room to store a block of data in FIFO. When both the FIFOs are full the host controller will stop the data coming from the card through Read Wait mechanism (if card supports Read Wait) or through clock stopping.
10	Wait for the Transfer Complete Interrupt and DMA Interrupt.
11	If Transfer Complete is set 1, go to Step(4) else if DMA Interrupt is set to 1, go to Step(12). Transfer Complete is higher priority than DMA Interrupt.
12	Write 1 to the DMA Interrupt in the Normal interrupt status register to clear this bit.
13	Set the next system address of the next data position to the System address register and go to Step (10).
14	Write 1 to the Transfer Complete and DMA Interrupt in the Normal interrupt status register to clear this bit.

Note: Step (2) and Step (3) can be executed at the same time. Step (5) and Step (6) can also be executed at the same time.

For example, if the host wants to transfer 4KB of data to the card. Assume the maximum block size is 512 bytes.

Then the host driver will program the Block size register as 512 and Block count register with the value 8.

The AHB master and transmitter residing inside the 3MCR host controller will get the information (how much data to transfer) from these registers. Using the above information,

the AHB master will act as a master and initiate a data Read transaction to read a block of data (512 bytes) from the system memory. Whenever a block of data is ready in FIFO, the transmitter will start transmitting the block of data (512) on SDMMC bus. After transmitting the entire block of data to the card, the transmitter will wait for a status response from the card. Transmitter will send the next block of data only, when it receives a good status response from the card for the previous block of data otherwise the transaction will be aborted and the host will go for a fresh transaction.

57.4.3 ADMA transactions

[Figure 1877](#) and [Table 1864](#) describe the ADMA transaction sequence.

Note: For data read/write using ADMA, swapping of bytes is not required. Swapping of bytes should be disabled using `EMMC_MSTR_EN_BYTE_SWAP` (bit 27) of `PCM REG3`. This bit should be reset to 0. Reset value of this bit is 1. SDMMC driver should reset it.

SD standard provides TCD value description in little endian format. To provide the correct value of TCD to HC these should be byte swapped in software before programming in SRAM memory.

Figure 1877. ADMA transaction flow

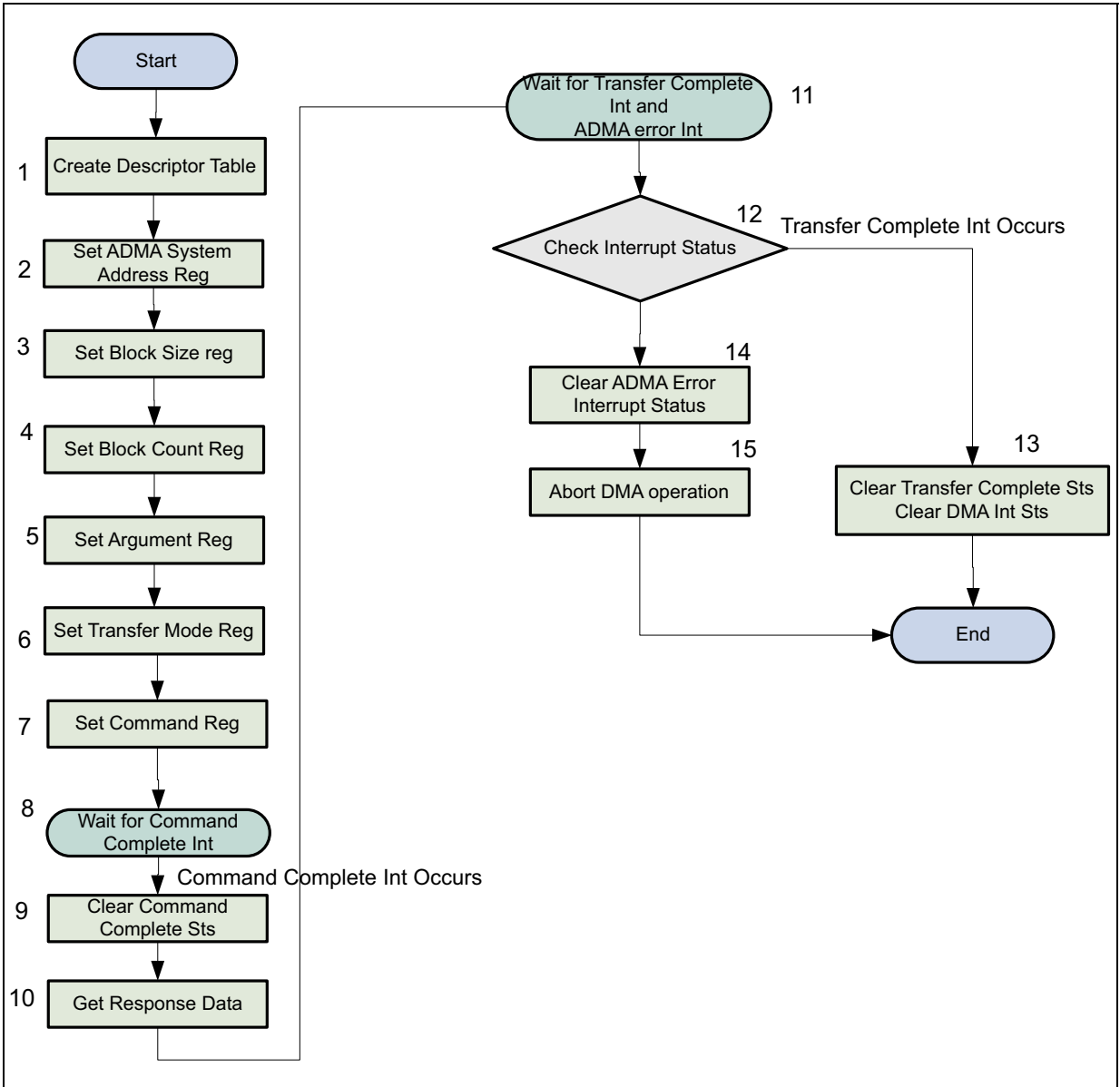


Table 1864. ADMA transaction flow

Step	Description
1	Create descriptor table for ADMA in the system memory.
2	Set the Descriptor address for ADMA in the ADMA System address register.
3	Set the value corresponding to the executed data byte length of one block in the Block size.

Table 1864. ADMA transaction flow (continued)

Step	Description
4	Set the value corresponding to the executed data block count in the Block count register. If the Block Count Enable in the Transfer mode register is set to 1, total data length can be designated by the Block count register and the Descriptor table. These two parameters indicate same data length. However, transfer length is limited by the 16-bit Block count register. If the Block Count Enable in the Transfer mode register is set to 0, total data length is designated by not Block count register but the Descriptor table. In this case, ADMA reads more data than length programmed in descriptor from SDMMC card. Too much read operation is aborted asynchronously and extra read data is discarded when the ADMA is completed
5	Set the argument value to the Argument register.
6	Set the value to the Transfer mode register. The host driver determines Multi / Single Block Select, Block Count Enable, Data Transfer Direction, Auto CMD12 Enable and DMA Enable.
7	Set the value to the Command register. When writing to the upper byte [3] of the Command register, the SDMMC command is issued and DMA is started.
8	Wait for the Command Complete Interrupt.
9	Write 1 to the Command Complete in the Normal interrupt status register to clear this bit.
10	Read Response register and get necessary information of the issued command.
11	Wait for the Transfer Complete Interrupt and ADMA Error Interrupt.
12	If Transfer Complete is set 1, go to Step (13) else if ADMA Error Interrupt is set to 1, go to Step (14).
13	Write 1 to the Transfer Complete Status in the Normal interrupt status register to clear this bit.
14	Write 1 to the ADMA Error interrupt status in the Error interrupt status register to clear this bit.
15	Abort ADMA operation. SDMMC card operation is stopped by issuing Abort command. If necessary, the host driver checks ADMA Error status register to detect why ADMA error is generated.

Note: Step (3) and Step (4) can be executed simultaneously. Step (6) and Step (7) can also be executed simultaneously.

57.4.4 Abort transaction (sync and async)

An Abort transaction is performed using CMD12 for a SDMMC memory card.

There are two cases where the HD needs to do an Abort transaction:

1. When the HD stops infinite block transfers.
2. When HD stops transfers while a multiple block transfer is exacting.

There are two ways to issue an Abort command. The first is an asynchronous Abort. The second is a synchronous Abort. In an asynchronous abort sequence, the HD can issue an Abort command at anytime unless command inhibit (CMD) in the Present state register is

set to 1. In a synchronous abort, the HD issues an Abort command after the data transfer stopped by using Stop at block gap request in the Block gap control register.

57.4.4.1 Synchronous abort

The flow for synchronous abort is shown in [Figure 1878](#) and [Table 1865](#).

Figure 1878. Synchronous abort transaction

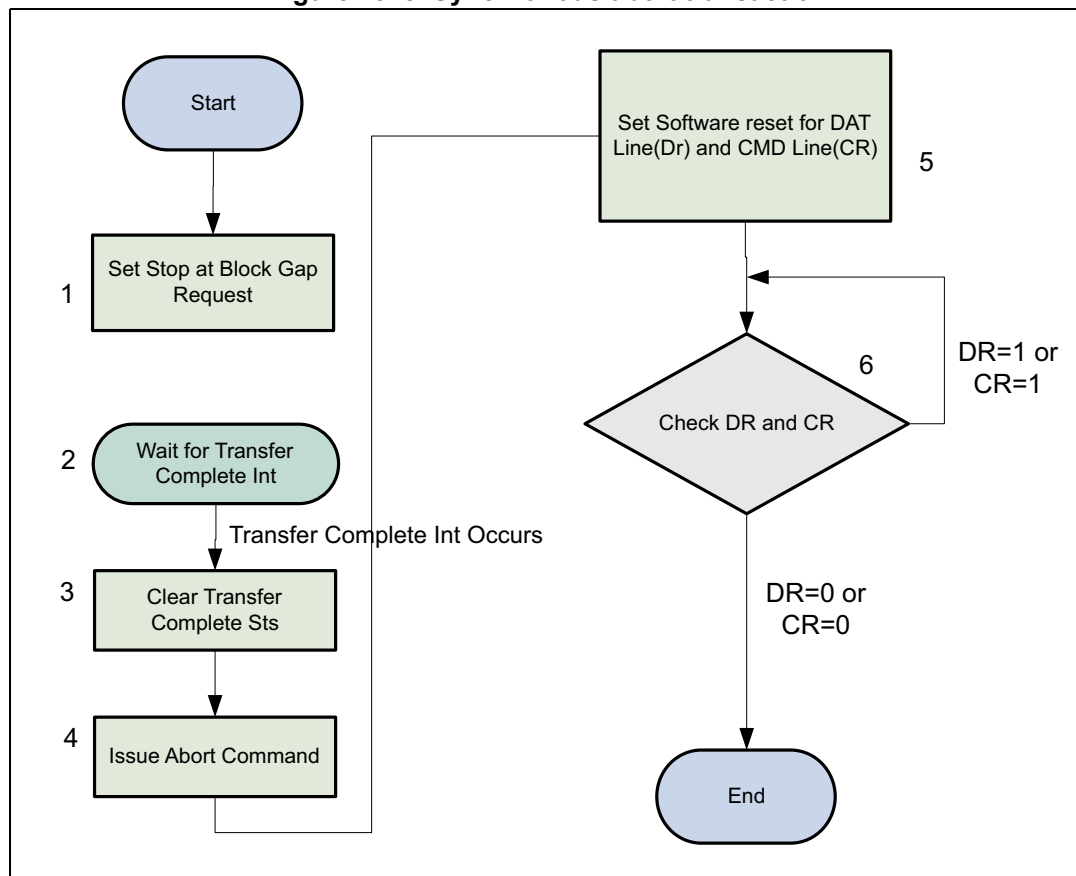


Table 1865. Abort transaction

Step	Description
1	Set the Stop at block gap request in the Block gap control register to 1 to stop SDMMC transactions.
2	Wait for Transfer Complete Interrupt.
3	Set the Transfer Complete to 1 in the Normal interrupt status register to clear this bit.
4	Issue Abort command.
5	Set both Software Reset for DAT Line and Software Reset for CMD Line to 1 in the Software reset register to do software reset.
6	Check Software Reset for DAT Line and Software Reset for CMD Line in the Software reset register. If both Software Reset for DAT Line and Software Reset for CMD Line are 0, go to "END". If either Software Reset for DAT Line or Software Reset for CMD Line is 1, repeat step(6).

57.4.4.2 Asynchronous abort

The flow for asynchronous abort is shown in [Figure 1879](#) and [Table 1866](#).

Figure 1879. Asynchronous abort

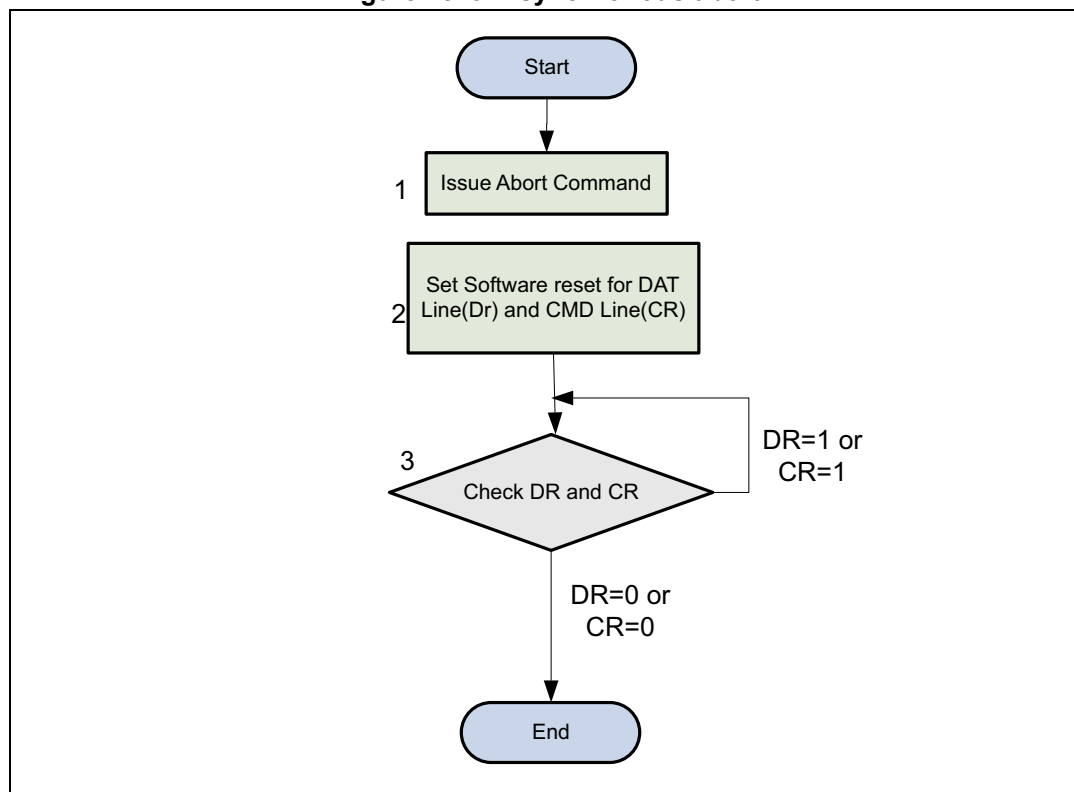


Table 1866. Asynchronous abort

Step	Description
1	Issue Abort command.
2	Set both Software Reset for DAT Line and Software Reset for CMD Line to 1 in the Software reset register to do software reset.
3	Check Software Reset for DAT Line and Software Reset for CMD Line in the Software reset register. If both Software Reset for DAT Line and Software Reset for CMD Line are 0, go to "End". If either Software Reset for DAT Line or Software Reset for CMD Line is 1, go to step (3).

57.4.5 Boot operation (normal and alternate)

The boot operation is described below.

Note: SoC does not support Execute In Place from eMMC.

57.4.5.1 Normal boot operation

The host driver writes the boot timeout value into Boot timeout control register as per the eMMC4.3+ specification.

When the host driver sets the “boot_en” to 1 in Block gap control register and data_trans_direction bit to “1” in Transfer mode register, the host controller drives the cmd line to “0” for boot operation.

If the host controller is configured to wait for boot ack from the eMMC4.3+ device, the controller receives the boot acknowledgment and asserts the “boot_ack_rcv” interrupt to the driver.

If the host controller does not receive the boot acknowledgment from the device within the timeout value, the host controller will assert the data timeout error interrupt.

After servicing the boot acknowledge interrupt or data timeout error interrupt, the driver programs the Boot timeout control register with boot data timeout value.

The eMMC4.3+ device starts sending the boot data on the data line and host controller sends the same to system whenever a block of data is received from device.

The host controller terminates the boot operation when the programmed number of blocks is transferred to the system. The driver can write “boot_en” as “0” in Block gap control register and then driver waits for boot terminate interrupt. After the boot termination interrupt comes, the host driver issues soft reset for DATA line to the host controller to reset the data state machines to idle state.

The boot operation can also be terminated in between the boot transfer. Program the Stop at block gap request while transfer is going on and then wait for Transfer Complete Interrupt. Once the driver receives the Transfer Complete Interrupt the driver clears the boot enable and then the host driver issues soft reset for DATA line to the host controller to reset the data state machines to idle state.

There is no need to perform error recovery sequence in case data timeout interrupt occurs during boot operation (that is, ignore sending Abort command, soft reset and just clear the timeout interrupt and proceed with the boot flow).

If the device sends wrong acknowledgment to the host, the host controller will assert data_crc_err interrupt on the driver. In this case, the host driver has to stop the boot mode operation by setting boot_en as “0” in Block gap control register and write soft reset for cmd and data line.

If the device sends end bit as “0” in acknowledgment to the host, the host controller will assert data_end_bit_err interrupt on the driver.

The host driver stops the boot mode operation by setting boot_en as “0” in Block gap control register and write soft reset for cmd and data line.

Note: Enable the boot_ack_chk bit during boot operation. If failed, the controller will not wait for boot acknowledge from the card and send out data CRC error when the card sends boot ack first and followed by boot data.

57.4.5.2 Alternate boot operation

The host driver writes the boot timeout value as per the eMMC4.3+ spec into Boot timeout control register.

If the “alt_boot_en” and “boot_en” is set to 1 in Block gap control register and “data_trans_direction” bit is set to 1 in Transfer mode register, the host controller drives the CMD0 (0xFFFFF0) on the CMD line.

The system waits for Command Complete Interrupt before “boot_ack_rcv” interrupt.

If the host controller is configured to wait for boot ack from the eMMC4.3+ device, the controller receives the boot acknowledgment and asserts the "boot_ack_rcv" interrupt to the driver.

The eMMC4.3+ device starts sending the boot data on the data line and host controller sends the same to system whenever a block of data is received from device.

The host controller terminates the boot operation when the programmed number of blocks are transferred to the system and the driver programs "boot_en" and alt_boot_en as "0" in Block gap control register and then wait for boot terminate interrupt. Once the driver receives the boot termination interrupt. The driver needs to send CMD0 to inform the device about the boot operation complete. The driver programs Command register with argument "0x00000000" for the cmd0 (CMDTYPE as Abort) and waits for Command Complete. After that the driver need to issues the soft reset for data to host controller to reset the data state machines to idle state.

The boot operation can be terminated at any time. Program the Stop at block gap request while transfer is going on and then wait for Transfer Complete Interrupt. Once the driver receives the Transfer Complete Interrupt the driver clears the boot_en and alt_boot_en and then the host driver needs to send CMD0 to inform the device about the boot operation complete. The host driver programs Command register with argument "0x00000000" for the cmd0 (CMDTYPE as Abort) and waits for Command Complete. After that the driver need to issues the soft reset for data to host controller to reset the data state machines to idle state.

If the host controller does not receive the acknowledgment from the device with in timeout value the host controller will assert the data timeout error interrupt.

The driver programs the Boot timeout control register with boot data timeout value.

There is no need to perform error recovery sequence in case of data timeout interrupt as mentioned above.

If the device sends wrong acknowledgment to the host, then host controller will assert data_crc_err interrupt to the driver. The host driver has to stop the boot mode operation by setting alt_boot_en and boot_en as "0" in Block gap control register and write soft reset for cmd and data line.

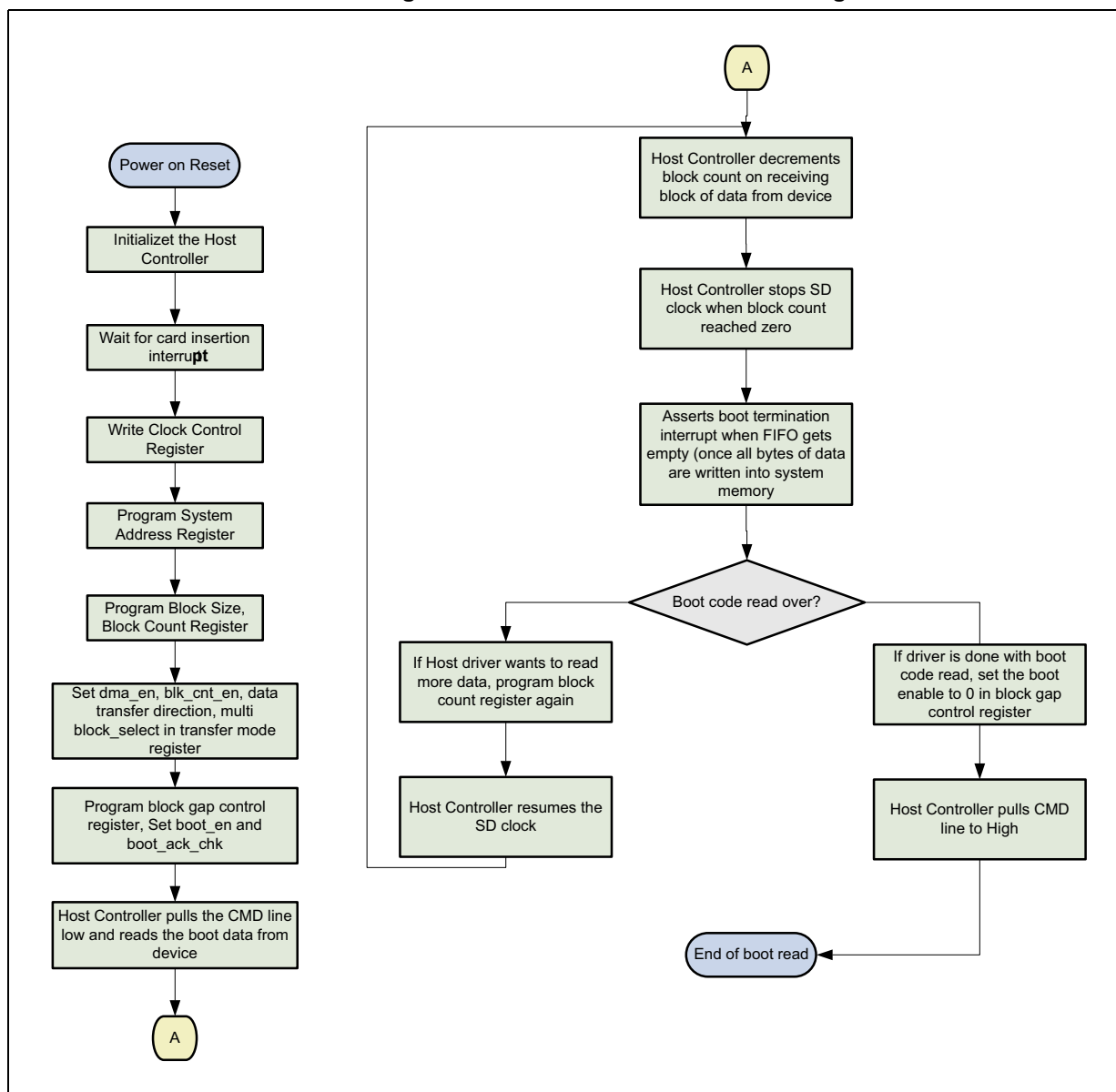
If the device sends end bit as "0" in acknowledgment to the host, the host controller asserts data_end_bit_err interrupt on the driver. The host driver stops the boot mode operation by setting alt_boot_en and boot_en as "0" in Block gap control register and write soft reset for cmd and data line.

Note: Enable the boot_ack_chk bit during boot operation. If failed, the controller will not wait for boot acknowledge from the card and send out data CRC error when the card sends boot ack first and followed by boot data.

57.4.5.3 Boot code chunk read operation

The following figure explains the boot code read done as chunks of data (that is the driver can read the boot data as 2 Kbytes, 4 Kbytes, 1 Kbytes and 8 Kbytes instead of 128 Kbytes at a time).

Figure 1880. Boot code access flow diagram



57.5 Timing

This section gives the details about various timing diagrams of MMC. Some text and figures are used from the MMC Specification version 4.51.

57.5.1 Timing diagram

Various timing diagrams are listed below.

Figure 1881. Multiple-block write timing

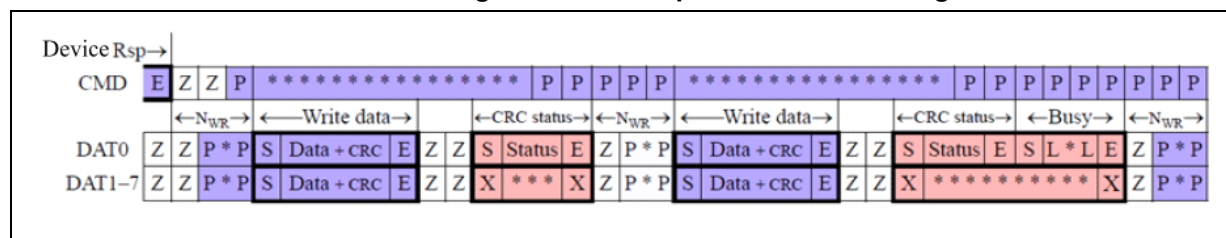


Figure 1882. Multiple block read

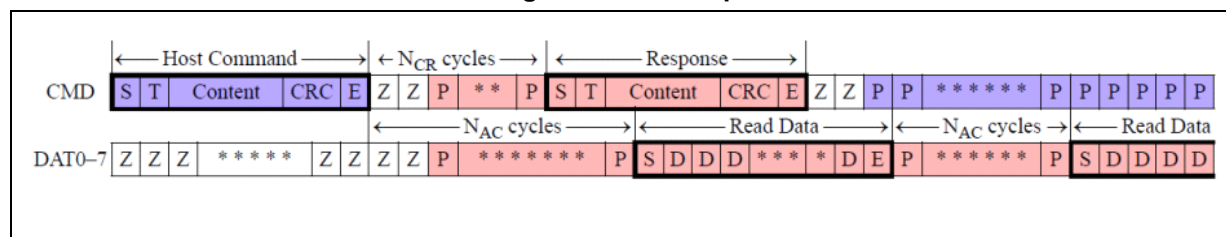


Figure 1883. Boot operation timing with termination between consecutive data blocks

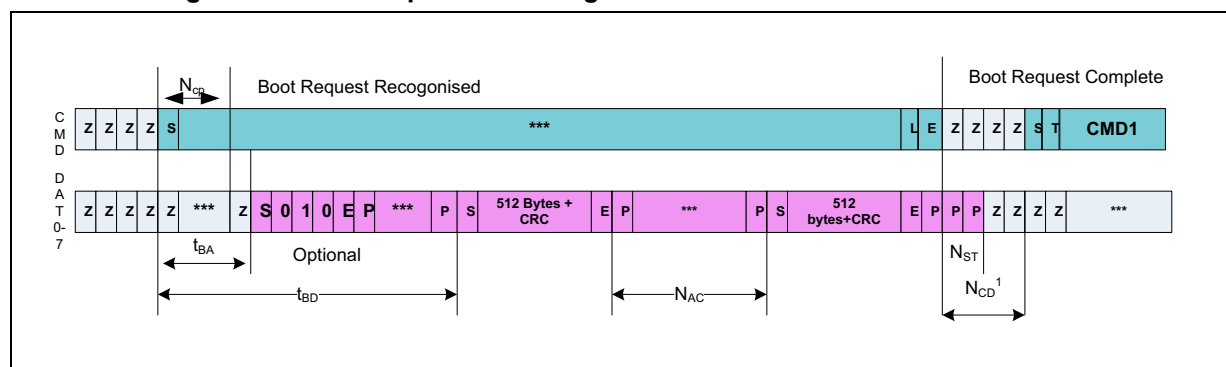


Figure 1884. Boot operation timing with termination during transfer

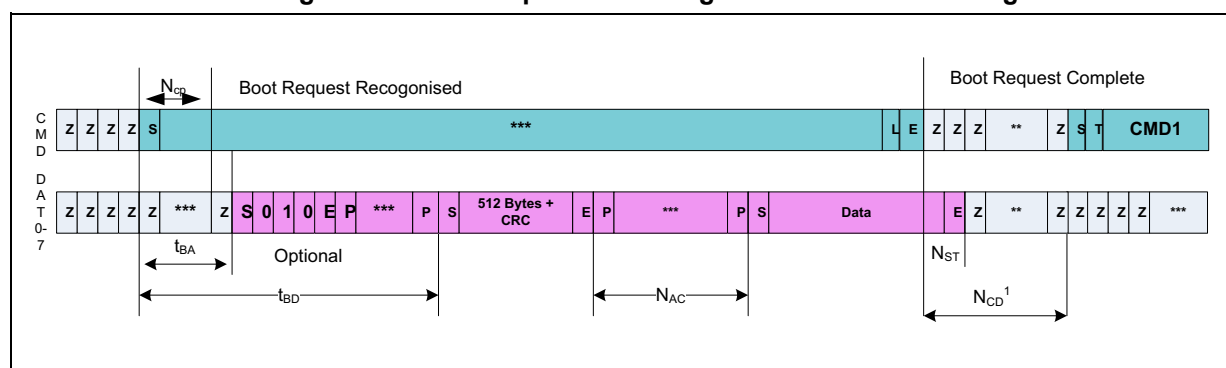
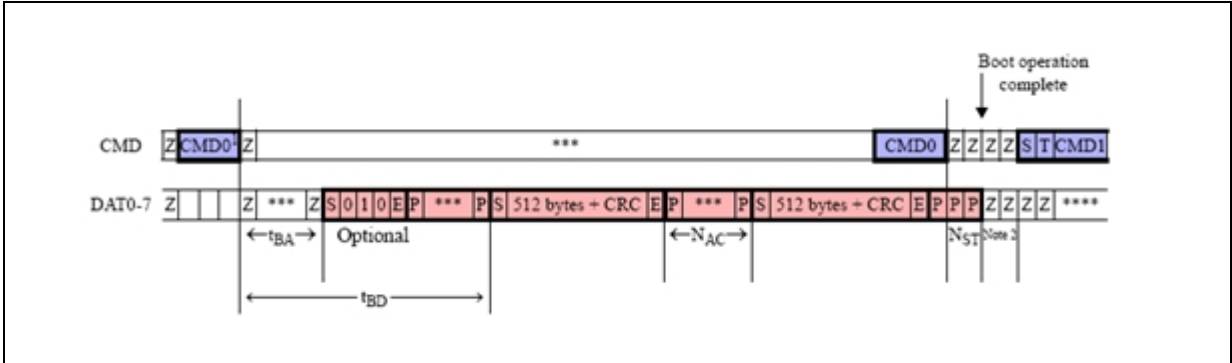
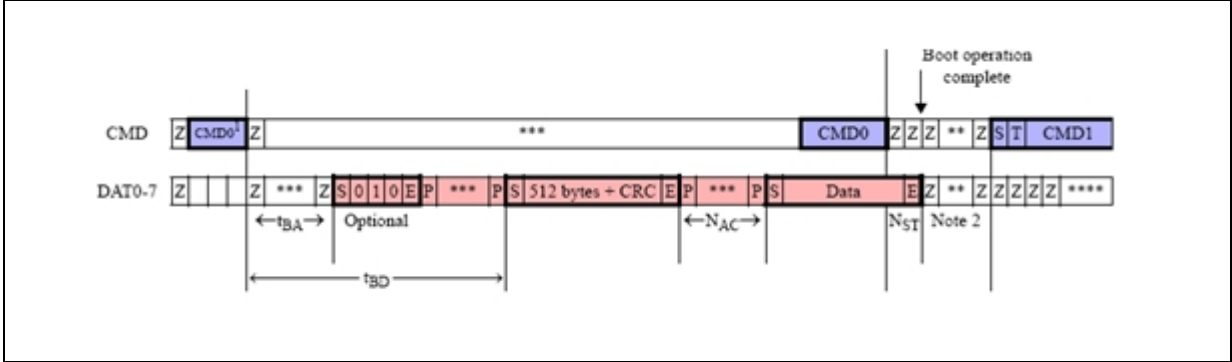


Figure 1885. Alternative boot operation, termination between consecutive data blocks



Note: CMD0 with argument 0xFFFFFFFF

Figure 1886. Alternative boot operation, termination during transfer



Note: CMD0 with argument 0xFFFFFFFF